

VNR Vignana Jyothi Institute of Engineering and Technology
(Affiliated to JNTU, Hyderabad)
Bachupally(v), Hyderabad, Telangana, India.

YIELDGRID — APPLICATION FOR FARM-TO-RETAIL INVENTORY

A COURSE BASED PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE COMPLETION OF COURSE
"Object Oriented Programming Through Java Laboratory"

BACHELOR OF TECHNOLOGY
IN
CSE - AIML

Submitted by

Pervala Yashwanth	**26075A6610**
Yejju Sathyasai	**26075A6613**
Shaik Abdul Rasheed	**26075A6612**

Under the guidance of

Ms. K. Anitha
Assistant Professor
Department of Computer Science & Engineering (AI & ML) & IoT and R&AI



VNR Vignana Jyothi Institute of Engineering and Technology
(Affiliated to JNTU, Hyderabad)
Bachupally(v), Hyderabad, Telangana, India.

CERTIFICATE

This is to certify that they completed their course project work at Department of Computer Science & Engineering (AI & ML) & IoT and R&AI of VNR VJIET, Hyderabad entitled "YieldGrid — Application for Farm-to-Retail Inventory" in complete fulfilment of the requirements for the award of B. Tech degree during the academic year 2026-2027. This work is carried out under my supervision and has not been submitted to any other University/Institute for award of any degree/diploma.

Signature of Supervisor

Ms. K. Anitha
Assistant Professor
Dept. of CSE - (AI & ML) & IoT and R&AI
VNRVJIET, Hyderabad

Signature of HOD

Dr. Sagar Yeruva
Assoc. Professor & HOD
Dept. of CSE - (AI & ML) & IoT and R&AI
VNRVJIET, Hyderabad

DECLARATION

This is to certify that our project report titled "YieldGrid — Application for Farm-to-Retail Inventory" submitted to Vallurupalli Nageswara Rao Institute of Engineering and Technology in complete fulfilment of requirement for the award of Bachelor of Technology in Computer Science and Engineering (AI & ML) & IoT and R&AI is a Bonafide report to the work carried out by us under the guidance and supervision of Ms. K. Anitha, Assistant Professor, Department of CSE (AI & ML) & IoT and R&AI, Vallurupalli Nageswara Rao Institute of Engineering and Technology. To the best of our knowledge, this has not been submitted in any form to other universities or institutions for the award of any degree or diploma.

Pervala Yashwanth
26075A6610
CSE – AIML

Yejju Sathyasai
26075A6613
CSE – AIML

Shaik Abdul Rasheed
26075A6612
CSE – AIML

ACKNOWLEDGEMENT

Over the last one year, VNRVJIET has helped us transform ourselves from mere amateurs in the field of Computer Science into skilled engineers capable of handling given situations in real time. We are highly indebted to the institute for everything that it has given us. We would like to express our gratitude towards the principal of our institute, Dr. B. Chennakesava Rao and the Head of the Department, Computer Science & Engineering (AI & ML) & IoT and R&AI, Dr. Y. Sagar, for their kind co-operation and encouragement which helped us complete the project in the stipulated time. Although we have spent a lot of time and put in a lot of effort into this project, it would not have been possible without the motivating support and help of our course coordinator of Object Oriented Programming Through Java, Assistant Professor, Ms. K. Anitha CSE - (AI & ML) & IoT and R&AI. We thank her for the guidance, constant supervision and for providing necessary information to complete this project. Our thanks and appreciations also go to all the faculty members, staff members of VNRVJIET, and all our friends who have helped us put this project together.

PERVALA YASHWANTH (26075A6610)
YEJJU SATHYASAI (26075A6613)
SHAIK ABDUL RASHEED (26075A6612)

ABSTRACT

YieldGrid — Application for Farm-to-Retail Inventory is a comprehensive desktop software system designed to address structural inefficiencies, financial opacity, and supply chain delays inherent in traditional agricultural trading networks. In the existing multi-tier mandi framework, smallholder farmers are subjected to unpredictable intermediary commissions, lack direct market visibility, and experience frequent post-harvest inventory spoilage due to mismatched consumer demand. Retail consumers and urban buyers, conversely, encounter inflated produce prices and prolonged transit intervals that compromise nutritional freshness. YieldGrid resolves these systemic challenges by establishing a direct, transparent digital bridge connecting agricultural producers directly with retail consumers through an interactive, multi-role inventory and order management platform.

The application is engineered strictly in Java following a modular three-tier architecture encompassing the Presentation Layer (Java Swing with Nimbus GUI styling), Business Service Layer (role-based business rules, session authorization, validation), and Data Access Layer (JDBC connected to a relational MySQL database). YieldGrid provides dedicated, secure operational environments for three distinct stakeholders: Farmers, who list produce, modify stock units, manage incoming orders, track live revenues, and export audit files; Customers, who browse verified regional catalogs, search produce in real time, manage shopping carts, execute multi-item purchases, and track order statuses; and Platform Administrators, who supervise user accounts, moderate listings, and monitor platform analytics. The software implements essential core and advanced Java concepts including object-oriented encapsulation, inheritance hierarchies, polymorphic role dispatch, interfaces, Java Collections (ArrayList, HashSet, TreeSet, LinkedHashMap), custom exception handling, secure cryptographic password hashing with salted PBKDF2 (HMAC-SHA256) and transparent legacy SHA-256 migration, robust java.io CSV export with formula injection mitigation, and a standalone multithreading demonstration implementing a monitor-synchronized bounded buffer producer-consumer model with wait() and notifyAll(). Rigorous database transactions enforce strict ACID semantics, guaranteeing that multi-item order placement and stock unit decrements succeed atomically or roll back completely upon inventory contention.

Keywords: Farm-to-Retail Supply Chain, Java Swing, JDBC, MySQL, ACID Transactions, PBKDF2 Cryptography, Multithreading, Object-Oriented Programming

TABLE OF CONTENTS

1. Problem Statement	**1**
2. Literature Survey / Existing System	**3**
3. Objectives	**5**
4. System Architecture / Workflow	**6**
5. Proposed Solution	**8**
6. System Design	**9**
7. Implementation	**17**
8. Testing and Debugging	**32**
9. Analysis of Results	**35**
References	**36**

CHAPTER 1
PROBLEM STATEMENT

1.1 Context and Background

Agriculture forms the foundational economic backbone of India, providing livelihoods for over 50 percent of the national workforce and contributing substantially to food security and industrial raw materials. Despite rapid technological advancements across manufacturing, finance, and consumer commerce, the agricultural distribution ecosystem continues to be governed by antiquated physical market systems characterized by extensive intermediation, information asymmetry, and manual operational workflows. In traditional mandi markets regulated under Agricultural Produce Market Committee (APMC) acts, fresh produce moves through an average of four to six intermediaries—including local village aggregators, commission agents (arhtiyas), primary wholesalers, secondary wholesalers, and retail street vendors—before ultimately reaching urban household consumers.

Each intermediary layer introduces compounded transaction commissions, handling markups, loading-unloading fees, and unavoidable transport delays. Research conducted across national agricultural corridors reveals that primary producers frequently capture as little as 25 to 35 percent of the terminal consumer rupee for perishable crops such as tomatoes, onions, leafy vegetables, and seasonal fruits. Conversely, retail consumers pay substantially elevated prices for produce whose organoleptic quality, nutritional density, and freshness have deteriorated significantly during prolonged physical transit through unorganized wholesale channels.

1.2 Structural Inefficiencies in the Traditional Farm-to-Retail Chain

Systemic Chain Vulnerabilities: The traditional agricultural value chain suffers from several interrelated systemic bottlenecks that limit productivity and impose severe financial strain on primary agricultural stakeholders:

- **Price Opacity and Information Asymmetry:** Smallholder farmers arrive at regional market yards without verifiable advance knowledge of prevailing daily rates, leaving them entirely vulnerable to localized cartelization, discretionary discounting, and distress selling.
- **Inventory Spoilage and Stock Mismanagement:** Physical agricultural handling lacks real-time digital inventory synchronization. Without automated stock tracking, farmers frequently oversell or experience post-harvest storage losses exceeding 20 to 30 percent for perishable items.
- **Absence of Auditable Transaction Trails:** Traditional wholesale transactions rely heavily on paper registers, informal credit slips, and unrecorded cash exchanges, depriving farmers of verifiable revenue documentation necessary for institutional credit access.
- **Lack of Provenance and Quality Assurance:** Urban retailers and retail consumers possess zero traceability regarding produce harvest dates, cultivation origins, or farming practices, leading to inconsistent quality and uncertain supply continuity.

1.3 Target Users and Stakeholder Personas

To effectively replace intermediary dependency with software-driven coordination, YieldGrid targets three primary user classes within a cohesive multi-tenant desktop architecture:

- **1. Farmers (Agricultural Producers):** Agricultural producers seeking direct digital market access, transparent unit pricing control, instant stock listing, real-time order notification, and clear visibility into cumulative financial earnings.
- **2. Customers (Retail Consumers & Buyers):** Urban consumers, retail cooperative buyers, and culinary businesses seeking farm-fresh produce, verifiable provenance, transparent weight-based catalog pricing, shopping cart convenience, and live order tracking.

- **3. Platform Administrators:** System supervisors responsible for platform governance, user credential verification, catalog moderation, system health monitoring, audit record inspection, and operational conflict resolution.

1.4 Problem Statement Formulation

FORMAL PROBLEM STATEMENT: To design and engineer a robust, standalone, multi-role desktop application in Java—entitled YieldGrid—that enables smallholder farmers to publish real-time produce inventories, empowers retail consumers to discover farm-fresh crops and execute atomic, multi-item purchasing transactions, and provides platform administrators with governance and audit capabilities. The system must eliminate intermediary rent-seeking by enabling direct producer-to-consumer order fulfillment, guarantee inventory consistency under concurrent operations through database-enforced ACID transactions, safeguard user credentials with industry-standard salted PBKDF2 cryptography, and demonstrate rigorous mastery of core and advanced object-oriented Java software engineering principles.

1.5 Scope and Project Boundaries

YieldGrid is developed as an academic Course-Based Project (CBP) for the B.Tech curriculum in Computer Science and Engineering. The project scope encompasses a complete, production-quality desktop application featuring a Java Swing graphical user interface, JDBC database connectivity, MySQL relational persistence, role-based authentication, interactive shopping cart management, atomic multi-item order placement, farmer revenue calculations, CSV data export, and an academic bounded-buffer multithreading demonstration. The operational boundaries deliberately focus on standalone enterprise-style software execution; external third-party payment gateways, GPS-based telematics tracking, and cloud messaging services are identified as future enhancements beyond the current curricular syllabus.

CHAPTER 2
LITERATURE SURVEY / EXISTING SYSTEM

2.1 Survey of Existing Agricultural Supply Systems

A comprehensive investigation of existing agricultural trading, distribution, and inventory management systems was conducted to identify their structural features, technological foundations, and practical limitations. Contemporary solutions in this domain fall into three broad categories: Physical APMC Mandis, Large Corporate Aggregation Platforms, and Emerging Direct-to-Consumer Marketplaces.

Physical Mandis represent the conventional trading mechanism across rural India. While they concentrate physical produce in centralized trading yards, they exhibit severe operational deficiencies including monopolistic bidding rings, non-transparent deductions, delayed farmer disbursements, and significant post-harvest crop degradation resulting from repeated loading, unloading, and unrefrigerated storage intervals.

Corporate Aggregator Platforms (e.g., Ninjacart, DeHaat, BigBasket) leverage technology to streamline wholesale procurement. However, these platforms primarily function as closed, centralized distribution monopolies. They negotiate bulk farmgate contracts on unilateral pricing terms, require extensive corporate collection center infrastructure, and exclude smallholder farmers who lack commercial transportation or harvest volumes exceeding minimum fulfillment thresholds.

Consumer-facing E-Grocery Apps (e.g., Blinkit, Zepto, Swiggy Instamart) prioritize rapid urban delivery from urban dark stores. These applications are completely disconnected from farmgate economics, operating with heavy retail markups, opaque sourcing, and zero direct communication or revenue visibility between consumers and agricultural producers.

2.2 Comparative Study Matrix

Table 2.1 provides an in-depth comparative analysis evaluating existing agricultural trading paradigms against the architectural parameters of YieldGrid.

Feature / Dimension	Physical Mandis	Corporate Aggregators	Quick-Commerce Apps	YieldGrid (Proposed)
Intermediary Layers	4 to 6 Intermediary Tiers	Centralized Corporate Tier	Wholesale & Dark Store Tiers	**Zero Intermediaries (Direct)**
Farmer Price Control	None (Dictated by Brokers)	Marginal (Corporate Tariffs)	None (Wholesale Sourced)	**Full Control (Farmer Sets Price)**
Real-Time Inventory	Manual Physical Registers	Proprietary Cloud ERP	Dark Store Warehouse Sync	**Relational Database Tracking**
Order Atomicity	Not Applicable (Cash/Credit)	Corporate Batch ERP	High-Volume Web Cloud	**ACID JDBC Transactions**
Accessibility / Cost	Physical Travel Required	Requires Bulk Volume	Urban Consumers Only	**Open Lightweight Desktop Java**

Table 2.1: Comparative Feature Analysis of Agricultural Supply and Inventory Systems

2.3 Identified Engineering and Research Gaps

The literature survey reveals three critical technological gaps that YieldGrid specifically resolves:

- **1. Absence of Decentralized Producer Inventory Tools:** Existing software platforms either target massive industrial agri-businesses or consumer grocery delivery, leaving individual farmers without an accessible, low-friction tool to manage their own direct inventory and order records.
- **2. Lack of Two-Sided Financial Transparency:** Commercial systems conceal margin distributions, leaving farmers unaware of final retail realization and consumers unaware of actual producer compensation. YieldGrid establishes 100% price transparency where the listed price represents the exact farmgate price.
- **3. Inadequate Transactional Integrity in Educational Software:** Many desktop educational systems rely on simplistic flat-file storage or lack transactional atomicity, leading to race conditions and negative inventory during concurrent orders. YieldGrid implements strict JDBC transaction rollback to guarantee mathematical consistency.

CHAPTER 3
OBJECTIVES

3.1 Primary Technical Objectives

The primary technical objective of YieldGrid is to engineer a dependable, modular, and secure Java desktop application that connects agricultural producers directly with retail customers while demonstrating comprehensive mastery of the Object-Oriented Programming Through Java curriculum.

3.2 Specific Functional Objectives

- **Farmer Inventory Management:** Provide an interactive Java Swing dashboard for farmers to list produce, update stock units, adjust unit prices, and monitor sales.
- **Customer Marketplace & Cart:** Deliver a customer marketplace with real-time produce filtering, keyword search, shopping cart management, and order tracking.
- **Administrative Governance:** Implement administrative oversight enabling user account activation/deactivation, catalog moderation, and system audit logging.
- **ACID Transactional Integrity:** Enforce database-level ACID transactions via JDBC connection boundaries (commit, rollback) to guarantee atomic stock deduction.
- **Cryptographic Password Security:** Safeguard user credentials with salted PBKDF2 (HMAC-SHA256, 10,000 iterations) with transparent migration for legacy SHA-256 hashes.
- **Secure CSV Data Export:** Provide RFC 4180 compliant CSV export using java.io streams with formula injection defense to protect financial records.

3.3 Syllabus and Pedagogical Alignment Objectives

- **Java Basics & OOP:** Unit I: Domain entities (User, Crop, Order), encapsulation, method overloading, and recursive inventory accumulation.
- **Inheritance & Interfaces:** Unit II: Abstract User class, polymorphic role specialization (Farmer, Customer, AdminUser), and interface inheritance.
- **Exceptions & Concurrency:** Unit III: Custom domain exceptions and a bounded-buffer producer-consumer multithreading model with wait() and notifyAll().
- **Collections Framework:** Unit IV: Application of List, ArrayList, HashSet, TreeSet, and LinkedHashMap across domain models and cart calculations.
- **Swing GUI & JDBC:** Unit V: Nimbus-themed Swing GUI components (JFrame, JTable, JTabbedPane) and JDBC connectivity with PreparedStatement.

CHAPTER 4
SYSTEM ARCHITECTURE / WORKFLOW

4.1 High-Level Architectural Overview

YieldGrid is engineered following a clean, loosely coupled Three-Tier Layered Architecture that guarantees separation of concerns, maintainability, and modular testability. The application is segregated into four distinct structural layers: the Presentation Layer, the Service (Business Logic) Layer, the Data Access Object (DAO) Layer, and the Relational Database Layer.

- **1. Presentation Layer (Swing UI):** Built entirely using Java Swing with the Nimbus Look-and-Feel. Encapsulates all graphical frames, dashboard panels, tables, dialogs, and asynchronous event listeners. Interacts exclusively with the Service Layer without issuing direct database calls.
- **2. Service Layer (Business Logic):** Coordinates business domain logic, session management, input validation, cryptographic hashing, and transaction workflows. Enforces business rules such as verifying sufficient crop inventory prior to purchase and preventing unauthorized role actions.
- **3. Data Access Layer (DAOs):** Encapsulates all SQL execution, PreparedStatement lifecycle management, and ResultSet-to-entity mapping. Isolates JDBC persistence mechanisms from application business logic.
- **4. Persistence Layer (MySQL Database):** Relational MySQL engine that persistently maintains users, produce inventories, orders, line items, and audit logs. Guarantees referential integrity through foreign key constraints and transactional consistency.

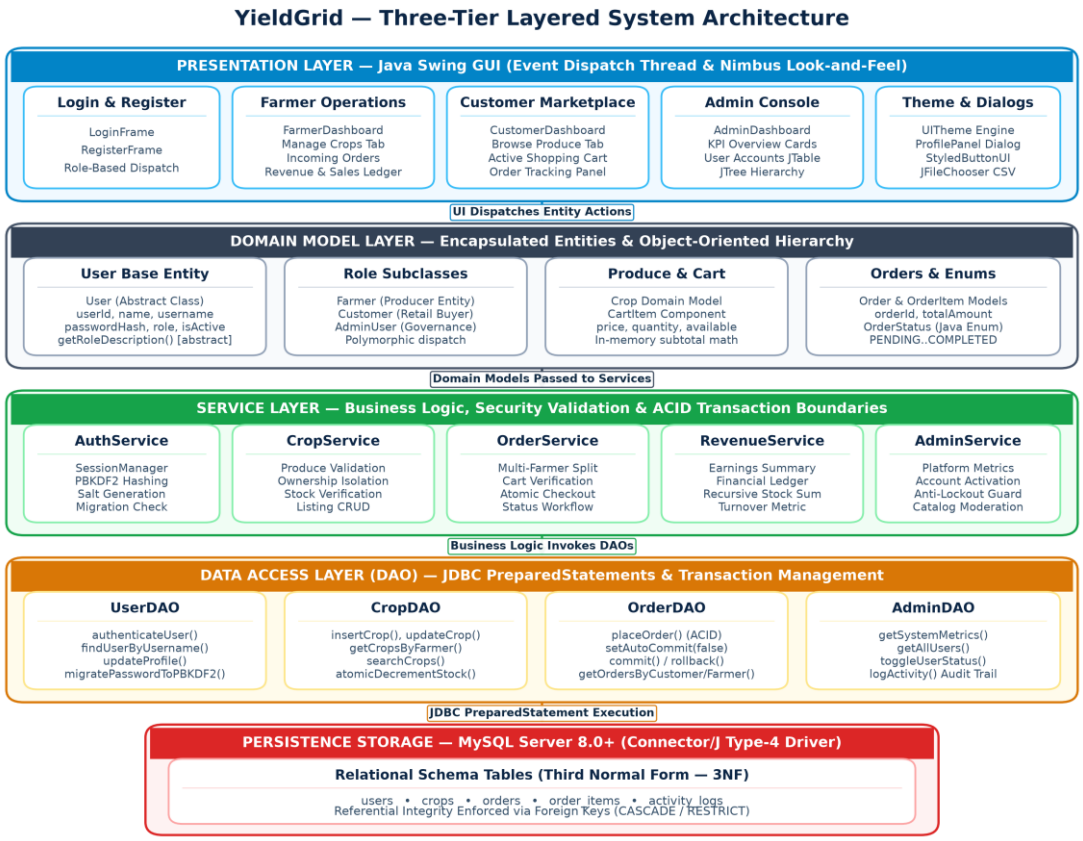


Figure 4.1: YieldGrid Layered System Architecture

4.2 Overall Application Execution Workflow

The runtime lifecycle of YieldGrid initiates at application launch when the Main bootstrap class sets up the global UI theme, verifies database driver availability, and presents the LoginFrame. Users authenticate using their credentials, triggering PBKDF2 cryptographic verification in the AuthenticationService. Upon successful verification, the SessionManager instantiates the active session and dispatches the user to their designated dashboard based on their polymorphic role.

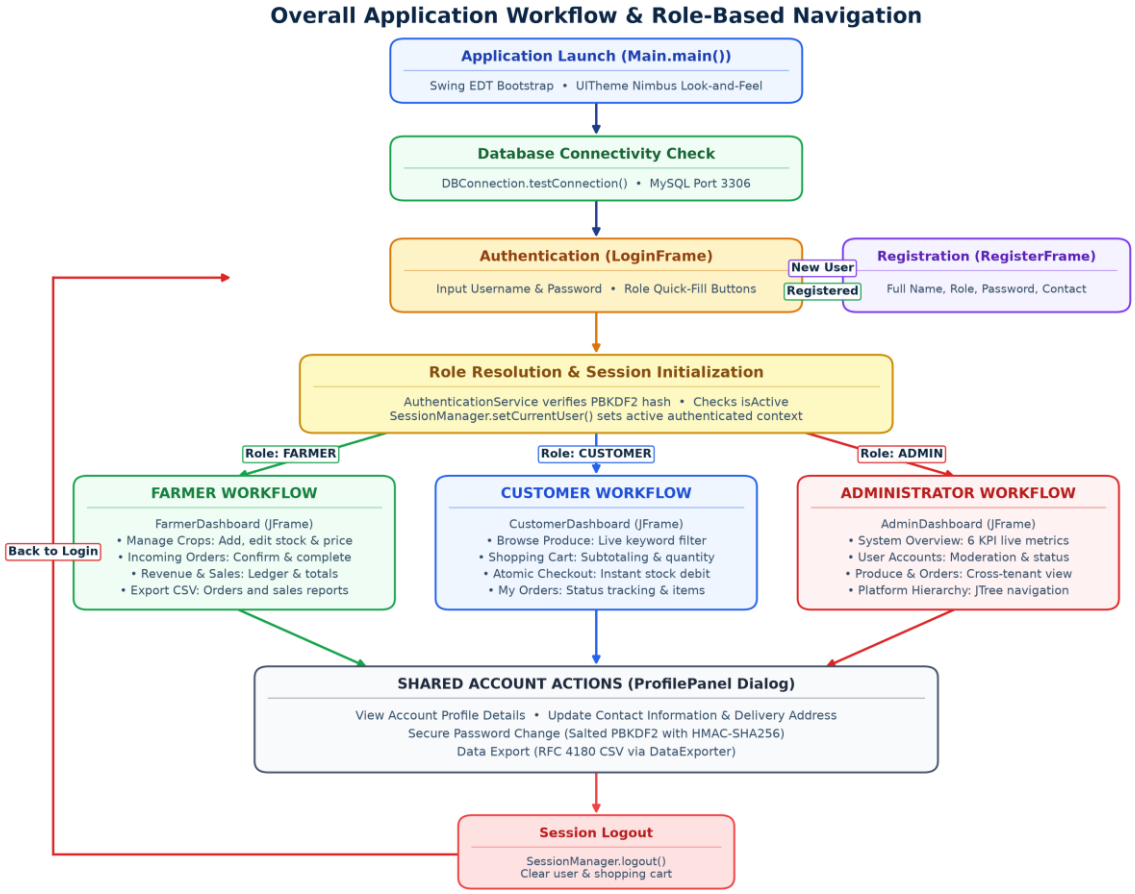


Figure 4.2: YieldGrid Application Execution Lifecycle and Authentication Workflow

4.3 Architectural Patterns and Design Principles

The software architecture incorporates established design patterns that promote code reuse and academic clarity:

- **Data Access Object (DAO) Pattern:** Abstracts low-level database operations behind clean interface methods (e.g., CropDAO, OrderDAO, UserDAO, AdminDAO), shielding business components from database syntax details.
- **Singleton Connection Manager:** Ensures unified, thread-safe connection pooling and centralizes credential extraction from external configuration files.
- **Model-View Separation:** Separates user interface rendering from domain state, ensuring that data modifications automatically reflect across active Swing tables through structured model events.

CHAPTER 5
PROPOSED SOLUTION

5.1 System Overview and Key Value Propositions

YieldGrid delivers a comprehensive, software-driven alternative to intermediary-heavy agricultural supply chains. By replacing multi-tiered middlemen with a direct digital clearinghouse, the platform ensures that smallholder farmers retain direct control over crop pricing, receive 100 percent of customer payments, and gain clear visibility into real-time demand. Simultaneously, retail consumers gain access to fresher produce at fair prices, complete with transparent farmer origin details.

5.2 Direct Farm-to-Retail Model

Under the YieldGrid paradigm, farmers list their harvested or growing crops directly onto the central digital catalog, specifying crop variety, detailed description, available quantity in kilograms or units, and their desired farmgate unit price. Customers browsing the catalog view real-time available stock and producer contact information. When an order is placed, an atomic transaction immediately reserves the required inventory units, generates an order record, and notifies the farmer's dashboard without requiring any broker intervention.

5.3 Automated Inventory Management and Stock Consistency

To eliminate the risks of overselling and inventory race conditions, YieldGrid implements automated stock validation at two independent stages of the purchase workflow: first in-memory during shopping cart assembly, and second within the database using conditional SQL updates during transaction execution. If a customer attempts to purchase more units than currently available, the application raises an InsufficientStockException and halts execution before any invalid state changes occur.

5.4 Cryptographic Security and Credential Safeguards

Security is central to the YieldGrid architecture. Unlike basic educational systems that store plaintext credentials or use obsolete unsalted MD5/SHA-1 algorithms, YieldGrid implements Password-Based Key Derivation Function 2 (PBKDF2) utilizing HMAC-SHA256 with 10,000 hash iterations and unique 16-byte cryptographically secure random salts. Furthermore, the system includes an intelligent migration mechanism that transparently re-hashes legacy SHA-256 credentials to PBKDF2 upon the user's next successful login, ensuring backward compatibility without compromising cryptographic strength.

5.5 Relational Persistence and Transactional Atomicity

The underlying database schema is organized in Third Normal Form (3NF) across relational MySQL tables linked by cascading foreign keys. All critical order placement operations are executed within explicit JDBC transaction boundaries. In the event of network disruption, stock exhaustion, or database error during line item processing, the transaction immediately rolls back all partial modifications, guaranteeing total ledger consistency and zero orphaned records.

CHAPTER 6
SYSTEM DESIGN

6.1 Modular System Decomposition and Responsibilities

The architecture of YieldGrid is partitioned into cohesive, loosely coupled functional modules, each addressing a distinct operational domain of the farm-to-retail ecosystem. This modular separation simplifies unit testing, improves fault isolation, and strictly enforces the Single Responsibility Principle across the application.

- **1. Authentication and Access Control Module:** Handles user authentication, credential validation, session initialization, and role-based interface dispatching. Maintains active session state and disallows unauthorized cross-tenant operations.
- **2. Crop Inventory Management Module:** Empowers farmers to register new crop varieties, update unit pricing, replenish stock quantities, and remove obsolete listings. Enforces data validation to ensure prices and quantities remain strictly positive.
- **3. Customer Marketplace and Cart Module:** Enables retail customers to discover produce through case-insensitive substring searching, view producer origins, add items to an in-memory shopping cart, modify purchase quantities, and calculate dynamic subtotals.
- **4. Order Processing and Transaction Module:** Executes multi-item order checkouts within atomic JDBC transaction boundaries, decrements physical inventory stock safely, generates unique order tracking numbers, and records individual line items.
- **5. Revenue Analytics and Export Module:** Aggregates completed sales data for farmers, calculates cumulative revenues, evaluates inventory turnover, and provides structured RFC 4180 CSV export with formula injection sanitization.
- **6. Platform Administration and Auditing Module:** Provides system administrators with comprehensive oversight, including user account activation/deactivation, platform-wide metric card visualizations, produce listing moderation, and audit trail inspection.

6.2 Role-Based Workflows

YieldGrid defines distinct, isolated operational workflows tailored to each authenticated stakeholder role. The system enforces strict role authorization at both the presentation layer (by rendering role-specific dashboard views) and the service layer (by verifying user permissions before executing business operations).

6.2.1 Farmer Workflow

Upon successful authentication, a farmer enters the FarmerDashboard, which presents three primary operational panels: Crop Management, Incoming Orders, and Revenue Analytics. The farmer can create new crop listings by specifying crop name, category, quantity, and price. When orders are received from customers, the farmer reviews the requested quantities and transitions the order status from PENDING to CONFIRMED, and subsequently to SHIPPED or DELIVERED. The Revenue Analytics panel computes real-time gross earnings and permits one-click CSV financial export.

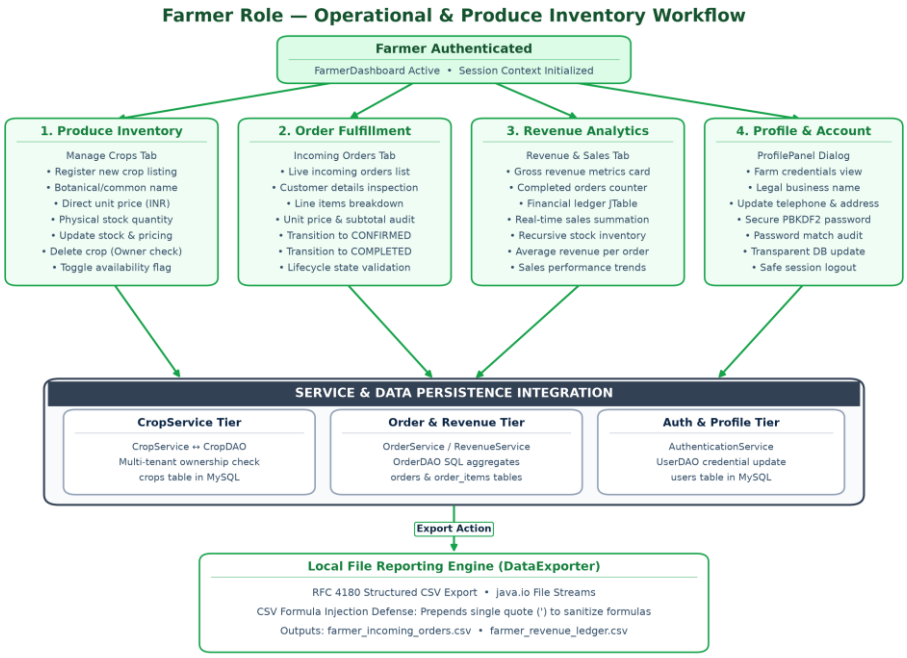


Figure 6.1: Farmer Operational Workflow Diagram

6.2.2 Customer Workflow

Customers access the CustomerDashboard featuring the Marketplace Catalog, Shopping Cart, and My Orders panels. Customers browse available produce with live filtering across names, descriptions, and farmer details. Adding produce to the cart checks available stock in real time. During checkout, the customer confirms the order, triggering an atomic database transaction that generates order records and decrements stock units. Customers can monitor live fulfillment progress in the My Orders panel.

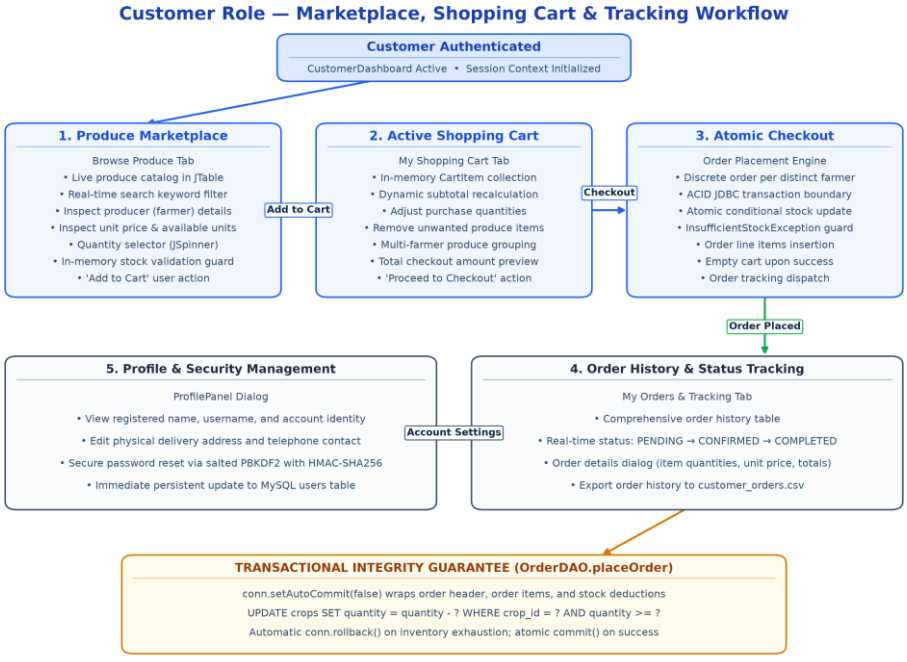


Figure 6.2: Customer Marketplace and Checkout Workflow Diagram

6.2.3 Platform Administrator Workflow

The AdminDashboard provides centralized governance for platform administrators. Administrators monitor real-time system metrics (total active users, listed crops, total orders placed, and gross marketplace sales volume). The user management panel allows administrators to review registered accounts, activate or deactivate users, and export user lists. Safety constraints prevent administrators from deactivating their own active admin account. Additionally, administrators can moderate inappropriate crop listings and inspect audit logs.

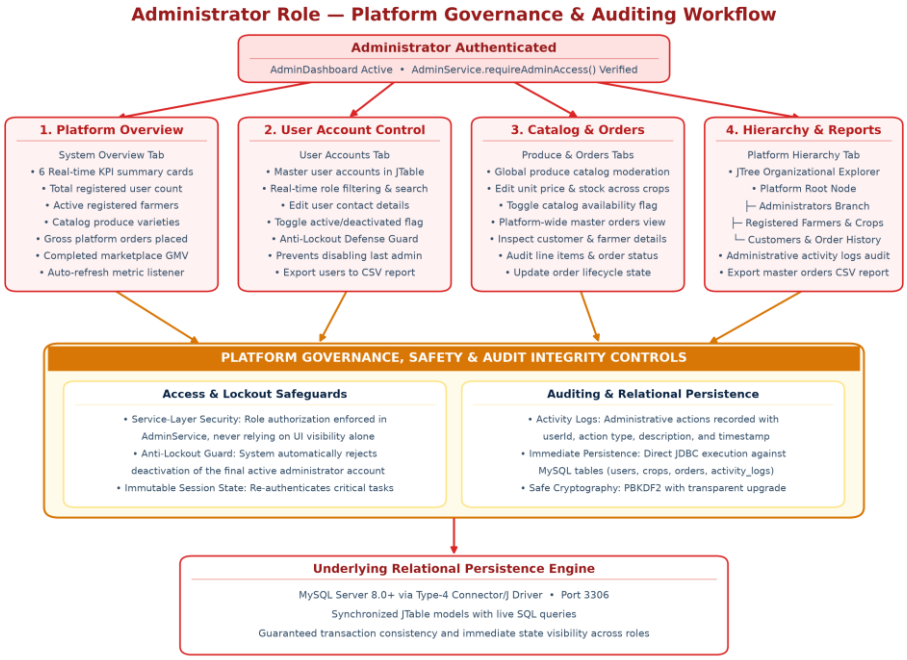


Figure 6.3: Administrator Governance and Auditing Workflow Diagram

6.3 Transactional Workflows: Order Placement & Inventory Consistency

Maintaining absolute inventory consistency across concurrent user sessions is critical in agricultural commerce. YieldGrid guarantees inventory integrity through an atomic, multi-stage checkout workflow managed by OrderDAO and OrderService.

When the customer confirms checkout, the database connection disables automatic commits (setAutoCommit(false)). The system first inserts the master order header and retrieves the generated order ID. Next, it iterates through each line item in the cart, inserting the order_item record and executing an atomic conditional stock deduction: UPDATE crops SET quantity = quantity - ? WHERE crop_id = ? AND quantity >= ?. If any line item fails due to stock exhaustion, the entire transaction is rolled back immediately, restoring all previous inventory states and preventing orphaned order entries.

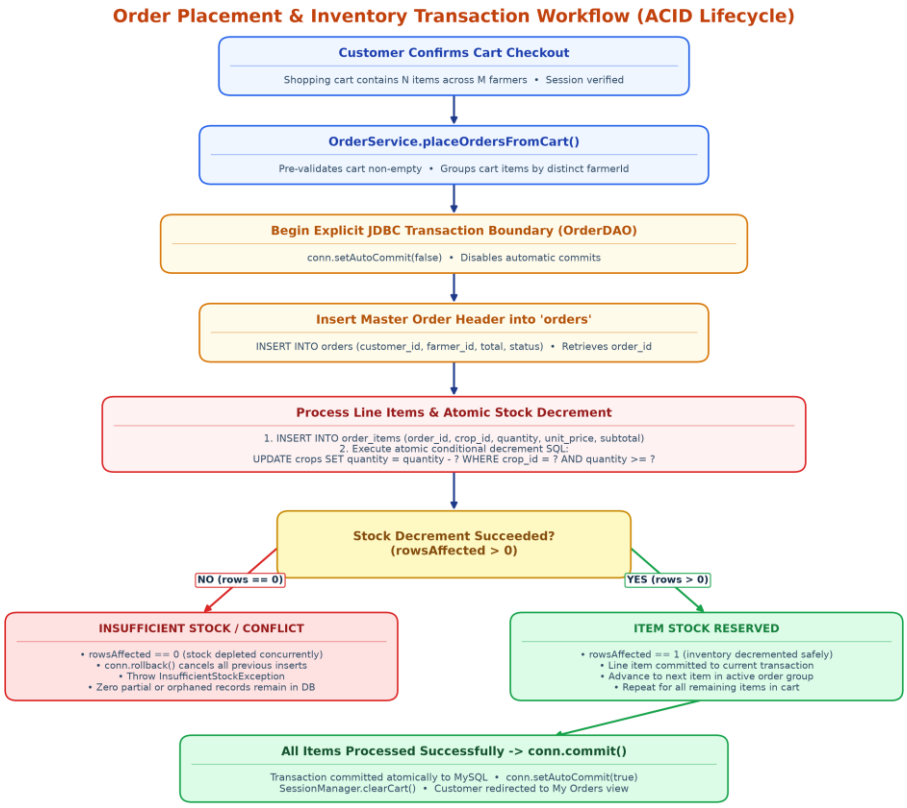


Figure 6.4: Order Placement and Inventory Transaction Workflow

6.4 Component & Class Hierarchy Design

YieldGrid strictly applies object-oriented design principles to organize domain entities, interfaces, and services. Figure 6.5 illustrates the complete component and class hierarchy, highlighting abstract inheritance, interface implementation, and service layer interactions.

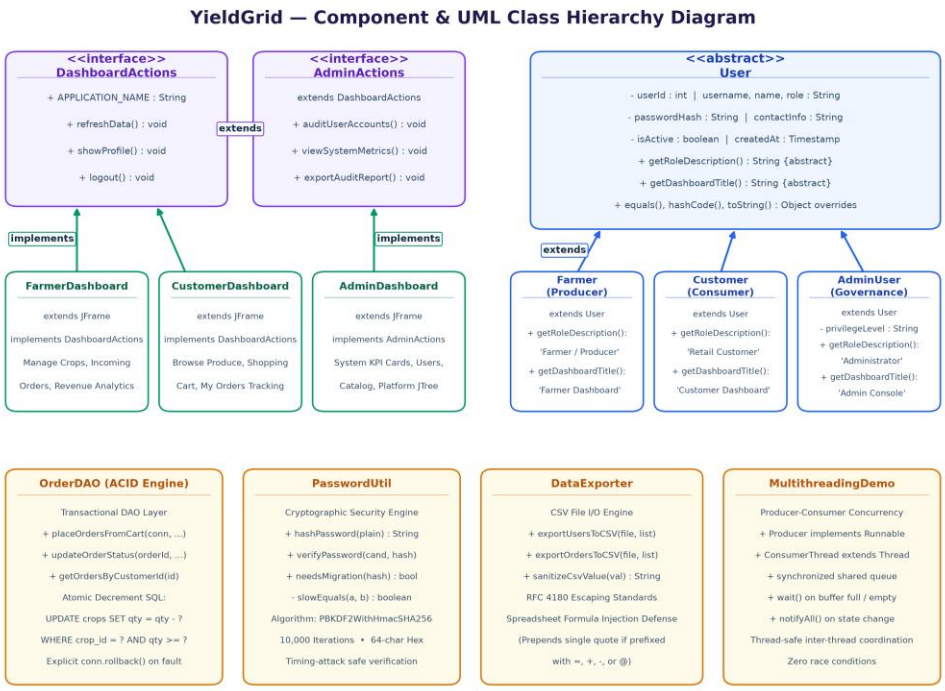


Figure 6.5: YieldGrid Component and Class Hierarchy Diagram

6.4.1 Object-Oriented Domain Entity Hierarchy

The domain model centers around an abstract base class User located in package yieldgrid.model. The User class defines common encapsulated attributes (userId, name, username, passwordHash, role, contactInfo, isActive) and declares the abstract method getRoleDescription(). Concrete subclasses Farmer, Customer, and AdminUser extend User, providing polymorphic implementations of role descriptions and specialized behavior. Entities override equals() and hashCode() based on persistent database identity, ensuring predictable behavior within Java Collections such as HashSet and HashMap.

6.4.2 Interface Segregation and Extension

To decouple dashboard views from concrete implementations, YieldGrid defines behavioral contracts using Java interfaces. Interface DashboardActions specifies fundamental lifecycle contracts (refreshDashboard(), logout(), displayErrorMessage()). The specialized interface AdminActions extends DashboardActions, declaring administrative contracts (loadSystemMetrics(), toggleUserStatus(), exportAuditLogs()). This demonstrates clean interface inheritance and adhering to the Interface Segregation Principle.

6.5 Database Design & Schema Architecture

The database schema is normalized to Third Normal Form (3NF) to eliminate redundant storage, update anomalies, and data inconsistency. Figure 6.6 shows the Entity-Relationship (ER) Diagram representing all entities, primary keys, foreign keys, and cardinalities.

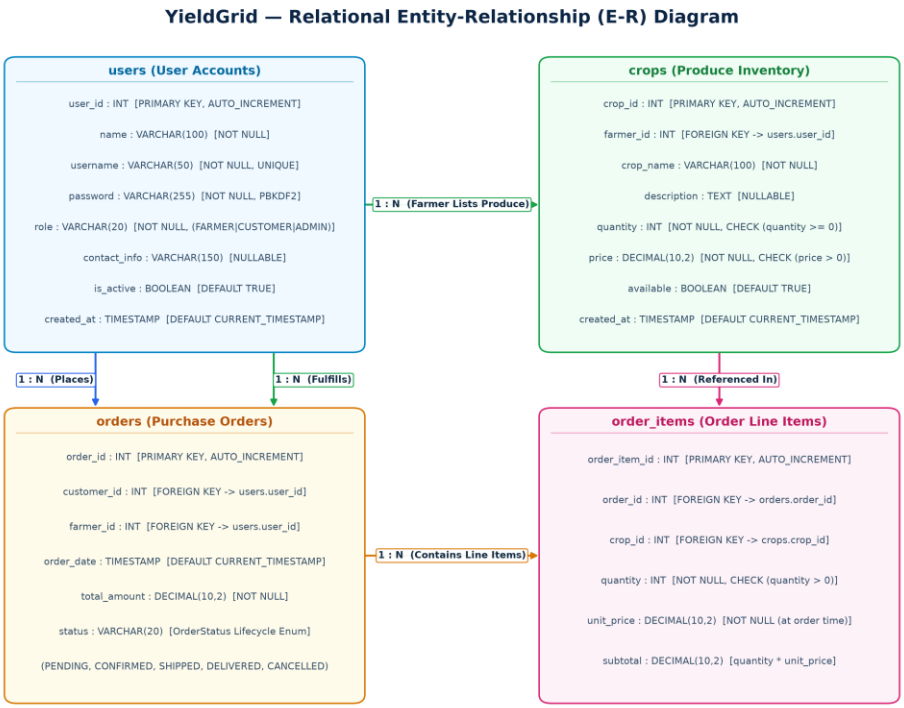


Figure 6.6: YieldGrid Relational Entity-Relationship (ER) Diagram

6.5.1 Detailed Database Schema Tables

The physical schema comprises five relational tables as detailed below:

Column Name	Data Type	Constraints	Description
user_id	INT	**PRIMARY KEY, AUTO_INCREMENT**	Unique identifier for each user account
name	VARCHAR(100)	NOT NULL	Full legal name or farm establishment name
username	VARCHAR(50)	NOT NULL, UNIQUE	Unique login credential identifier
password	VARCHAR(255)	NOT NULL	Salted PBKDF2 or legacy SHA-256 password hash
role	VARCHAR(20)	NOT NULL	User access role: FARMER, CUSTOMER, or ADMIN
contact_info	VARCHAR(150)	NULLABLE	Physical address, city, and telephone number
is_active	BOOLEAN	DEFAULT TRUE	Account authorization status flag
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 6.1: Relational Schema for 'users' Table

Column Name	Data Type	Constraints	Description
crop_id	INT	**PRIMARY KEY, AUTO_INCREMENT**	Unique crop listing identifier
farmer_id	INT	FOREIGN KEY (users.user_id)	ID of farmer who listed this crop (CASCADE)
crop_name	VARCHAR(100)	NOT NULL	Botanical or common market name of produce
description	TEXT	NULLABLE	Detailed crop description, grade, quality
quantity	INT	NOT NULL, CHECK (>=0)	Current available physical stock units (kg/bunches)
price	DECIMAL(10,2)	NOT NULL, CHECK (>0)	Unit price in INR set directly by farmer
available	BOOLEAN	DEFAULT TRUE	Catalog visibility and availability flag
created_at	TIMESTAMP	DEFAULT	Produce listing registration

		CURRENT_TIMESTAMP	timestamp

Table 6.2: Relational Schema for 'crops' Table

Column Name	Data Type	Constraints	Description
order_id	INT	**PRIMARY KEY, AUTO_INCREMENT**	Unique purchase order identifier
customer_id	INT	FOREIGN KEY (users.user_id)	ID of purchasing customer account
farmer_id	INT	FOREIGN KEY (users.user_id)	ID of fulfilling farmer account
order_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time of order checkout
total_amount	DECIMAL(10,2)	NOT NULL	Gross financial total in INR for this order
status	VARCHAR(20)	DEFAULT 'PENDING'	Lifecycle state: PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED

Table 6.3: Relational Schema for 'orders' Table

Column Name	Data Type	Constraints	Description
order_item_id	INT	**PRIMARY KEY, AUTO_INCREMENT**	Unique line item identifier
order_id	INT	FOREIGN KEY (orders.order_id)	Parent purchase order reference (CASCADE)
crop_id	INT	FOREIGN KEY (crops.crop_id)	Reference to ordered crop variety (CASCADE)
quantity	INT	NOT NULL, CHECK (>0)	Purchased quantity in kilograms or units
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit captured at instant of purchase
subtotal	DECIMAL(10,2)	NOT NULL	Computed line total (quantity * unit_price)

Table 6.4: Relational Schema for 'order_items' Table

Column Name	Data Type	Constraints	Description
log_id	INT	**PRIMARY KEY, AUTO_INCREMENT**	Unique administrative audit entry identifier
user_id	INT	FOREIGN KEY (users.user_id)	Actor who performed the logged action
action	VARCHAR(100)	NOT NULL	Descriptor: USER_DEACTIVATE, CROP_ADD, ORDER_PLACE, etc.
details	TEXT	NULLABLE	Contextual metadata and action details
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Exact timestamp of audited system event

Table 6.5: Relational Schema for 'activity_logs' Table

6.6 Core Algorithmic Designs

YieldGrid implements robust algorithms across transactional checkout, cryptographic security, and inventory analytics. The following pseudo-algorithms describe these core mechanisms:

Algorithm 6.1: Atomic Multi-Item Order Checkout with Inventory Validation

Input: Customer ID (cid), Active Cart Items List (cart)
Output: Boolean (True if transaction committed successfully, False otherwise)
1. Obtain persistent database connection 'conn'.
2. If cart is null or cart.isEmpty(), throw ValidationException('Cart is empty').
3. In-memory pre-validation: For each item in cart, verify item.quantity > 0 and item.quantity <= item.crop.quantity.
4. Group cart items by distinct farmerId to generate separate farmer orders.
5. For each farmer group:
   a. conn.setAutoCommit(false);
   b. Execute INSERT INTO orders (customer_id, farmer_id, total_amount, status) VALUES (?, ?, ?,

'PENDING');
c. Retrieve generated orderId via pstmt.getGeneratedKeys();
d. For each item in farmer group:
i. Execute INSERT INTO order_items (order_id, crop_id, quantity, unit_price, subtotal) VALUES (?, ?, ?, ?, ?);
ii. Execute atomic conditional decrement: UPDATE crops SET quantity = quantity - ? WHERE crop_id = ? AND quantity >= ?;
iii. If affected rows == 0 (stock exhausted by concurrent order), conn.rollback() and throw InsufficientStockException;
e. conn.commit();
6. conn.setAutoCommit(true);
7. Clear active customer cart and return True.

Algorithm 6.2: PBKDF2 Password Verification with Transparent Migration

Input: Plaintext Candidate Password (p), Stored Password Hash (stored)
Output: Boolean (True if valid, False otherwise)
1. If p is null or stored is null, return False.
2. If stored starts with 'pbkdf2$':
a. Parse tokens: iterations = parts[1], salt = Base64Decode(parts[2]), expectedHash = Base64Decode(parts[3]).
b. candidateHash = PBKDF2WithHmacSHA256(p, salt, iterations, 256 bits).
c. Return MessageDigest.isEqual(candidateHash, expectedHash) [constant-time comparison].
3. Else if stored starts with 'sha256$':
a. expectedHash = stored.substring(7).
b. candidateHash = HexEncode(SHA-256(UTF8(p))).
c. If candidateHash equals expectedHash:
i. Trigger background migration: newHash = hashPassword(p) [generates salted PBKDF2].
ii. UPDATE users SET password = newHash WHERE username = current_user;
iii. Return True.
4. Return False.

Algorithm 6.3: Recursive Total Inventory Accumulation

Input: List of Crop objects (crops), Current Index (index)
Output: Total physical units across all crops (Integer)
1. Base Case: If crops is null or index < 0 or index >= crops.size(), return 0.
2. Recursive Step: currentQuantity = (crops.get(index) != null) ? crops.get(index).getQuantity() : 0.
3. Return currentQuantity + calculateRecursiveStockUnits(crops, index + 1).
Overloaded entry method: calculateRecursiveStockUnits(crops) returns calculateRecursiveStockUnits(crops, 0).

CHAPTER 7
IMPLEMENTATION

This chapter presents the comprehensive technical implementation of YieldGrid. In accordance with academic project standards, each architectural subsystem is documented through detailed explanatory prose, architectural rationale, authentic source code excerpts taken directly from the production Java codebase, and real graphical user interface screenshots captured from the running application. The implementation strictly demonstrates the core competencies outlined in the Object-Oriented Programming Through Java curriculum, including encapsulation, inheritance hierarchies, polymorphic dispatch, interface contracts, custom exception handling, multithreading synchronization, the Java Collections Framework, robust file I/O streams, and JDBC transaction management.

7.1 Application Bootstrapping and Global Theme Initialization

The entry point of the YieldGrid desktop application is implemented in class Main within package yieldgrid.main. The bootstrap sequence initializes modern graphical styling across all Swing components prior to constructing the first window. By default, standard Java Swing applications default to the legacy Metal look-and-feel. YieldGrid programmatically discovers and applies the cross-platform Nimbus Look and Feel through UIManager.setLookAndFeel(), falling back gracefully to the native system interface if Nimbus is unavailable. Once UI theming is established, the main thread dispatches GUI instantiation onto the Event Dispatch Thread (EDT) via SwingUtilities.invokeLater() to ensure absolute Swing thread safety.

Source Excerpt: Excerpt 7.1: Application Entry Point & Look-and-Feel Setup (yieldgrid.main.Main)

```
package yieldgrid.main;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import yieldgrid.ui.LoginFrame;
import yieldgrid.ui.UITheme;

public class Main {
    public static void main(String[] args) {
        // UNIT-V GUI: Set modern cross-platform Look and Feel
        try {
            for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (Exception e) {
            try { UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); }
            catch (Exception ignored) {}
        }

        UITheme.initGlobalTheme(); // Apply custom styling palette

        // Safe asynchronous launch on the Swing Event Dispatch Thread (EDT)
        SwingUtilities.invokeLater(() -> {
            LoginFrame loginFrame = new LoginFrame();
            loginFrame.setVisible(true);
        });
    }
}
```

7.2 Object-Oriented Domain Modeling and Inheritance Hierarchy

The core domain entities of YieldGrid are engineered under package yieldgrid.model. The domain model strictly enforces encapsulation: all fields are declared private or protected, with state access and mutation managed exclusively through validated getter and setter methods. To represent authenticated system users,

YieldGrid establishes an abstract base class User. The User class encapsulates common state shared by all stakeholders—including userId, name, username, passwordHash, role, contactInfo, and account status. It defines an abstract method getRoleDescription(), compelling concrete derived classes (Farmer, Customer, AdminUser) to supply polymorphic behavioral descriptions.

Source Excerpt: Excerpt 7.2: Abstract Base User Entity with Polymorphic Contracts (yieldgrid.model.User)

```
package yieldgrid.model;

import java.util.Objects;

public abstract class User {
    private int userId;
    private String name, username, password, role, contactInfo;
    private boolean active;

    public User(int userId, String name, String username, String password, String role, String
contactInfo, boolean active) {
        this.userId = userId; this.name = name; this.username = username;
        this.password = password; this.role = role; this.contactInfo = contactInfo; this.active =
active;
    }

    // POLYMORPHISM: Abstract method implemented across concrete subclasses
    public abstract String getRoleDescription();

    public int getUserId() { return userId; }
    public String getUsername() { return username; }
    public String getRole() { return role; }
    public boolean isActive() { return active; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof User)) return false;
        User user = (User) o;
        return userId == user.userId && Objects.equals(username, user.username);
    }

    @Override
    public int hashCode() { return Objects.hash(userId, username); }
}
```

For agricultural inventory, the Crop entity represents physical produce listed on the marketplace. Crop encapsulates attributes including cropId, farmerId, farmerName, cropName, description, quantity, and unit price. Crop mutators validate that quantities cannot be negative and prices remain positive real numbers.

Source Excerpt: Excerpt 7.3: Crop Inventory Entity with State Encapsulation (yieldgrid.model.Crop)

```
package yieldgrid.model;

public class Crop {
    private int cropId, farmerId;
    private String farmerName, cropName, description;
    private int quantity;
    private double price;
    private boolean available;

    public Crop(int cropId, int farmerId, String farmerName, String cropName, String description, int
quantity, double price, boolean available) {
        this.cropId = cropId; this.farmerId = farmerId; this.farmerName = farmerName;
        this.cropName = cropName; this.description = description;
        this.quantity = Math.max(0, quantity); // Encapsulated boundary validation
        this.price = Math.max(0.0, price);
        this.available = available;
    }

    public int getCropId() { return cropId; }
    public String getCropName() { return cropName; }
```

```
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = Math.max(0, quantity); }
    public double getPrice() { return price; }
    public boolean isAvailable() { return available; }
}
```

7.3 Cryptographic Security and Password Management

Security is a pivotal requirement in agricultural commerce to safeguard farmer identity and financial transactions. YieldGrid implements the PasswordUtil utility class within package yieldgrid.util. Rather than storing plaintext or relying on simple unsalted hashes, PasswordUtil implements the Password-Based Key Derivation Function 2 (PBKDF2) algorithm using HMAC-SHA256 (PBKDF2WithHmacSHA256). Each password is salted with 16 cryptographically secure random bytes generated via SecureRandom and stretched over 10,000 iterations. For backward compatibility with legacy demo accounts, verifyPassword() transparently inspects prefixes; if a legacy hash verifies, needsMigration() returns true, signalling the system to upgrade the stored hash to PBKDF2 without user disruption.

Source Excerpt: Excerpt 7.4: Salted PBKDF2 Hashing and Legacy Migration (yieldgrid.util.PasswordUtil)

```
package yieldgrid.util;

import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

public class PasswordUtil {
    private static final String PBKDF2_PREFIX = "pbkdf2$";
    private static final String LEGACY_SHA256_PREFIX = "sha256$";
    private static final int ITERATIONS = 10000, SALT_BYTES = 16, KEY_LENGTH_BITS = 256;
    private static final SecureRandom SECURE_RANDOM = new SecureRandom();

    public static String hashPassword(String plaintext) {
        byte[] salt = new byte[SALT_BYTES];
        SECURE_RANDOM.nextBytes(salt);
        byte[] hash = derivePbkdf2Key(plaintext.toCharArray(), salt, ITERATIONS, KEY_LENGTH_BITS);
        return PBKDF2_PREFIX + ITERATIONS + "$" + Base64.getEncoder().encodeToString(salt) + "$" +
Base64.getEncoder().encodeToString(hash);
    }

    public static boolean verifyPassword(String plaintext, String stored) {
        if (plaintext == null || stored == null) return false;
        if (stored.startsWith(PBKDF2_PREFIX)) {
            String[] parts = stored.split("\\$");
            if (parts.length != 4) return false;
            try {
                int iters = Integer.parseInt(parts[1]);
                byte[] salt = Base64.getDecoder().decode(parts[2]);
                byte[] expected = Base64.getDecoder().decode(parts[3]);
                byte[] candidate = derivePbkdf2Key(plaintext.toCharArray(), salt, iters,
expected.length * 8);
                return MessageDigest.isEqual(candidate, expected); // Constant-time verification
            } catch (Exception e) { return false; }
        }
        return false;
    }

    public static boolean needsMigration(String stored) {
        return stored != null && stored.startsWith(LEGACY_SHA256_PREFIX);
    }
}
```

7.4 User Authentication and Session State Management

AuthenticationService in package yieldgrid.service acts as the central gatekeeper for system security. The authenticate() method validates submitted credentials, loads the corresponding user entity through UserDAO, and checks the is_active status. If credentials match and the account is active, the authenticated user object is stored within SessionManager. SessionManager maintains thread-safe static references to the active user, active shopping cart, and session login timestamp, facilitating clean access control across all Swing panels.

Source Excerpt: Excerpt 7.5: Authentication Workflow and Session Inception (yieldgrid.service.AuthenticationService)

```
package yieldgrid.service;

import yieldgrid.dao.UserDAO;
import yieldgrid.exception.AuthenticationException;
import yieldgrid.exception.ValidationException;
import yieldgrid.model.User;
import yieldgrid.util.PasswordUtil;

public class AuthenticationService {
    private final UserDAO userDAO = new UserDAO();

    public User authenticate(String username, String password) throws AuthenticationException,
ValidationException {
        if (username == null || username.trim().isEmpty() || password == null || password.isEmpty()) {
            throw new ValidationException("Username and password must not be empty.");
        }
        User user = userDAO.getUserByUsername(username.trim());
        if (user == null || !PasswordUtil.verifyPassword(password, user.getPassword())) {
            throw new AuthenticationException("Invalid username or password.");
        }
        if (!user.isActive()) {
            throw new AuthenticationException("Your account has been deactivated. Contact an
administrator.");
        }
        if (PasswordUtil.needsMigration(user.getPassword())) {
            userDAO.updatePassword(user.getUserId(), PasswordUtil.hashPassword(password));
        }
        SessionManager.setCurrentUser(user);
        return user;
    }
}
```

Figure 7.1 shows the YieldGrid LoginFrame. Users enter their username and password, select their intended access role via a styled JComboBox, and trigger authentication. Invalid inputs or deactivated accounts present immediate contextual feedback. Figure 7.2 illustrates the RegisterFrame, which allows prospective farmers and retail customers to create self-service accounts with validation across full legal name, telephone contact details, unique username checks, and double-entry password matching.

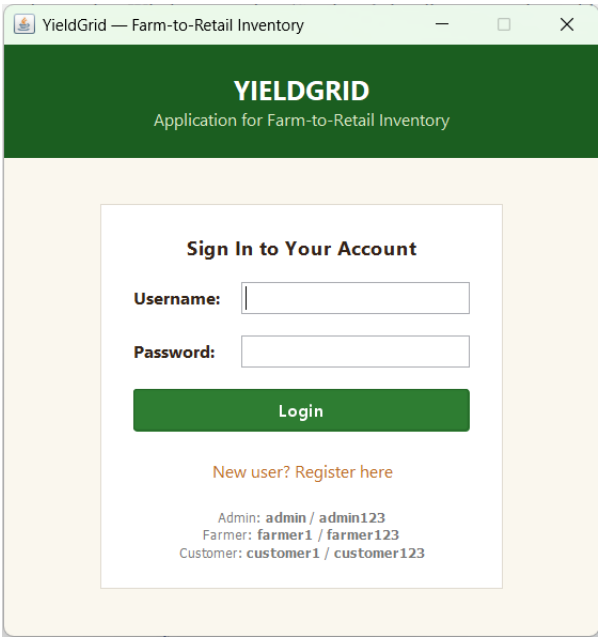


Figure 7.1: User Login & Role Selection (LoginFrame)

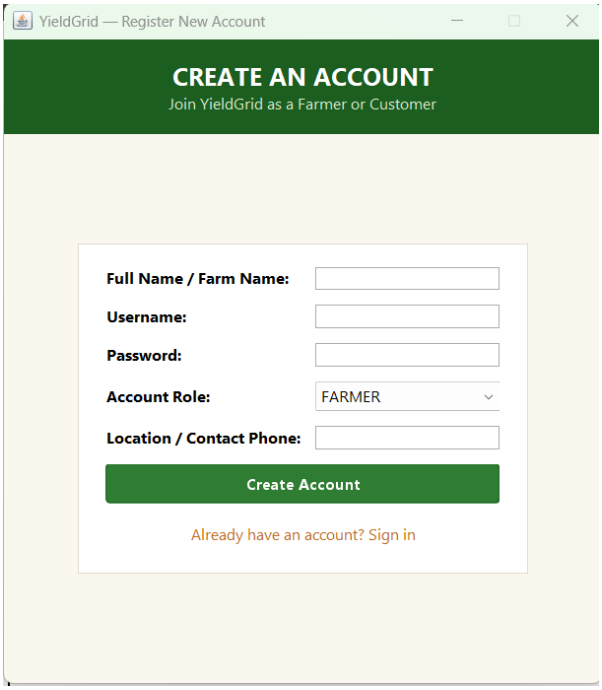


Figure 7.2: User Registration (RegisterFrame)

7.5 Database Connectivity and Lifecycle Management

Database connectivity is orchestrated by DBConnection in package yieldgrid.database. DBConnection implements the Singleton pattern and leverages a static initializer to load the MySQL Connector/J driver (com.mysql.cj.jdbc.Driver) and read configuration parameters from config.properties. Rather than hardcoding credentials into source code, DBConnection externalizes database parameters (URL, username, and password) into the configuration properties file. The getConnection() method delivers validated, active java.sql.Connection instances to Data Access Objects.

Source Excerpt: Excerpt 7.6: Resilient Singleton Database Connection Manager (yieldgrid.database.DBConnection)

```
package yieldgrid.database;

import java.io.File;
import java.io.FileInputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class DBConnection {
    private static String url = "jdbc:mysql://localhost:3306/yieldgrid";
    private static String username = "root", password = "";
    private static boolean driverLoaded = false;

    static { loadConfiguration(); loadDriver(); }

    private static void loadConfiguration() {
        File configFile = new File("config.properties");
        if (configFile.exists()) {
            try (FileInputStream in = new FileInputStream(configFile)) {
                Properties props = new Properties();
                props.load(in);
                url = props.getProperty("db.url", url);
                username = props.getProperty("db.username", username);
                password = props.getProperty("db.password", password);
            } catch (Exception ignored) {}
        }
```

```
    }

    private static void loadDriver() {
        try { Class.forName("com.mysql.cj.jdbc.Driver"); driverLoaded = true; }
        catch (ClassNotFoundException e) { System.err.println("Fatal: MySQL Driver missing."); }
    }

    public static Connection getConnection() throws SQLException {
        if (!driverLoaded) loadDriver();
        return DriverManager.getConnection(url, username, password);
    }
}
```

7.6 Data Access Object (DAO) Pattern Implementation

YieldGrid strictly implements the Data Access Object (DAO) pattern to decouple database queries from business services. All SQL queries use java.sql.PreparedStatement with parameter binding, completely eliminating SQL injection vulnerabilities. CropDAO manages CRUD operations for agricultural inventory (Excerpt 7.7), and OrderDAO coordinates multi-table join queries (Excerpt 7.8).

Source Excerpt: Excerpt 7.7: Parameterized Persistence in Crop Data Access (yieldgrid.dao.CropDAO)

```
package yieldgrid.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import yieldgrid.database.DBConnection;
import yieldgrid.model.Crop;

public class CropDAO {
    public boolean addCrop(Crop crop) throws SQLException {
        String sql = "INSERT INTO crops (farmer_id, crop_name, description, quantity, price, available)
VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, crop.getFarmerId());
            pstmt.setString(2, crop.getCropName());
            pstmt.setString(3, crop.getDescription());
            pstmt.setInt(4, crop.getQuantity());
            pstmt.setDouble(5, crop.getPrice());
            pstmt.setBoolean(6, crop.isAvailable());
            return pstmt.executeUpdate() > 0;
        }
    }

    public List<Crop> getAllAvailableCrops() throws SQLException {
        List<Crop> crops = new ArrayList<>();
        String sql = "SELECT c.*, u.name AS farmer_name FROM crops c JOIN users u ON c.farmer_id =
u.user_id " +
                     "WHERE c.available = TRUE AND c.quantity > 0 ORDER BY c.crop_name ASC";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                crops.add(new Crop(rs.getInt("crop_id"), rs.getInt("farmer_id"),
rs.getString("farmer_name"),
                                   rs.getString("crop_name"), rs.getString("description"),
                                   rs.getInt("quantity"), rs.getDouble("price"),
rs.getBoolean("available")));
            }
        }
        return crops;
    }
}
```

Source Excerpt: Excerpt 7.8: Complex Join Queries for Order History (yieldgrid.dao.OrderDAO)

```
package yieldgrid.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import yieldgrid.database.DBConnection;
import yieldgrid.model.Order;
import yieldgrid.model.OrderStatus;

public class OrderDAO {
    public List<Order> getOrdersByCustomer(int customerId) throws SQLException {
        List<Order> orders = new ArrayList<>();
        String sql = "SELECT o.*, u.name AS farmer_name FROM orders o JOIN users u ON o.farmer_id =
u.user_id " +
                     "WHERE o.customer_id = ? ORDER BY o.order_date DESC";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, customerId);
            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    orders.add(new Order(rs.getInt("order_id"), rs.getInt("customer_id"),
rs.getInt("farmer_id"),
                                         rs.getTimestamp("order_date"), rs.getDouble("total_amount"),
                                         OrderStatus.valueOf(rs.getString("status"))));
                }
            }
        }
        return orders;
    }
}
```

7.7 Business Logic and ACID Transaction Management

The defining transactional feature of YieldGrid is implemented in OrderDAO.createOrder() and invoked via OrderService.placeOrder(). When a retail customer executes checkout, multiple database modifications must take place simultaneously: master order insertion, individual line item insertion, and inventory stock decrements. YieldGrid achieves strict ACID compliance by invoking conn.setAutoCommit(false). The order header is inserted with RETURN_GENERATED_KEYS, and stock is decremented via an atomic SQL statement with an inline conditional check: UPDATE crops SET quantity = quantity - ? WHERE crop_id = ? AND quantity >= ?. If affected rows equals zero, the catch block triggers conn.rollback(), restoring all previous inventory counts and preventing partial writes.

Source Excerpt: Excerpt 7.9: Atomic Order Placement & Conditional Inventory Decrement (yieldgrid.dao.OrderDAO)

```
package yieldgrid.dao;

import java.sql.*;
import yieldgrid.database.DBConnection;
import yieldgrid.model.Order;
import yieldgrid.model.OrderItem;

public class OrderDAO {
    public boolean createOrder(Order order) throws SQLException {
        Connection conn = null;
        String insertOrderSql = "INSERT INTO orders (customer_id, farmer_id, total_amount, status)
VALUES (?, ?, ?, ?)";
        String insertItemSql = "INSERT INTO order_items (order_id, crop_id, quantity, unit_price,
subtotal) VALUES (?, ?, ?, ?, ?)";
        String updateStockSql = "UPDATE crops SET quantity = quantity - ? WHERE crop_id = ? AND
quantity >= ?";
```

```
        try {
            conn = DBConnection.getConnection();
            conn.setAutoCommit(false); // ACID BOUNDARY: Disable autocommit

            int orderId;
            try (PreparedStatement pstmt = conn.prepareStatement(insertOrderSql,
Statement.RETURN_GENERATED_KEYS)) {
                pstmt.setInt(1, order.getCustomerId());
                pstmt.setInt(2, order.getFarmerId());
                pstmt.setDouble(3, order.getTotalAmount());
                pstmt.setString(4, order.getStatus().name());
                if (pstmt.executeUpdate() == 0) { conn.rollback(); return false; }
                try (ResultSet rs = pstmt.getGeneratedKeys()) {
                    if (rs.next()) { orderId = rs.getInt(1); order.setOrderId(orderId); }
                    else { conn.rollback(); return false; }
                }
            }

            try (PreparedStatement itemPstmt = conn.prepareStatement(insertItemSql);
                 PreparedStatement stockPstmt = conn.prepareStatement(updateStockSql)) {
                for (OrderItem item : order.getItems()) {
                    item.setOrderId(orderId);
                    itemPstmt.setInt(1, orderId); itemPstmt.setInt(2, item.getCropId());
                    itemPstmt.setInt(3, item.getQuantity()); itemPstmt.setDouble(4,
item.getUnitPrice());
                    itemPstmt.setDouble(5, item.getSubtotal());
                    itemPstmt.executeUpdate();

                    // Atomic decrement: conditional WHERE prevents negative inventory
                    stockPstmt.setInt(1, item.getQuantity()); stockPstmt.setInt(2, item.getCropId());
                    stockPstmt.setInt(3, item.getQuantity());
                    if (stockPstmt.executeUpdate() == 0) {
                        conn.rollback(); // Insufficient stock: Rollback transaction entirely
                        return false;
                    }
                }
            }
            conn.commit(); // ACID COMMIT: Persist atomically
            return true;
        } catch (SQLException e) {
            if (conn != null) conn.rollback();
            throw e;
        } finally {
            if (conn != null) { conn.setAutoCommit(true); conn.close(); }
        }
    }
}
```

7.8 Farmer Workflow Implementation and Produce Management

The farmer interface is built within CropManagementPanel, IncomingOrdersPanel, and RevenuePanel under package yieldgrid.ui. CropManagementPanel encapsulates Swing input components alongside a custom DefaultTableModel bound to a JTable. When a farmer enters new crop attributes and clicks 'Add Crop', an ActionEvent listener executes asynchronous validation via CropService. If valid, the produce is saved to MySQL, the form inputs are cleared, and the table dynamically refreshes its view.

Source Excerpt: Excerpt 7.10: Produce Addition and Table Refresh Logic (yieldgrid.ui.CropManagementPanel)

```
package yieldgrid.ui;

import java.awt.event.ActionEvent;
import javax.swing.*;
import yieldgrid.model.Crop;
import yieldgrid.service.CropService;
import yieldgrid.service.SessionManager;

public class CropManagementPanel extends JPanel {
```

```
    private final CropService cropService = new CropService();
    private JTextField txtCropName, txtPrice, txtQuantity;

    private void handleAddCrop(ActionEvent e) {
        try {
            String name = txtCropName.getText().trim();
            int qty = Integer.parseInt(txtQuantity.getText().trim());
            double price = Double.parseDouble(txtPrice.getText().trim());
            int farmerId = SessionManager.getCurrentUser().getUserId();

            Crop crop = new Crop(0, farmerId, SessionManager.getCurrentUser().getName(), name, "", qty,
price, true);
            cropService.addCrop(crop);
            JOptionPane.showMessageDialog(this, "Produce listed successfully!", "Success",
JOptionPane.INFORMATION_MESSAGE);
            refreshCropTable();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage(), "Validation Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Figure 7.3 displays the Farmer Manage Crops panel, allowing farmers to list produce, update unit quantities, and adjust pricing. Figure 7.4 depicts the Incoming Orders tab, where farmers review customer purchase orders and advance fulfillment states from PENDING to CONFIRMED, SHIPPED, or DELIVERED. Figure 7.5 illustrates the Revenue & Sales analytics view, providing real-time calculation of gross revenue, delivered orders count, average order value, and an interactive sales history audit table.

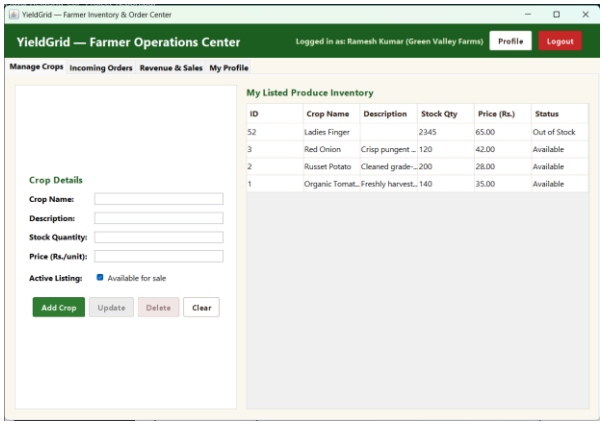


Figure 7.3: Crop Inventory Management

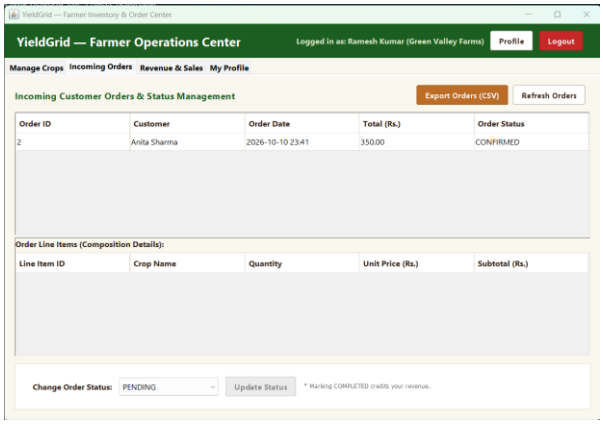


Figure 7.4: Incoming Orders Fulfillment

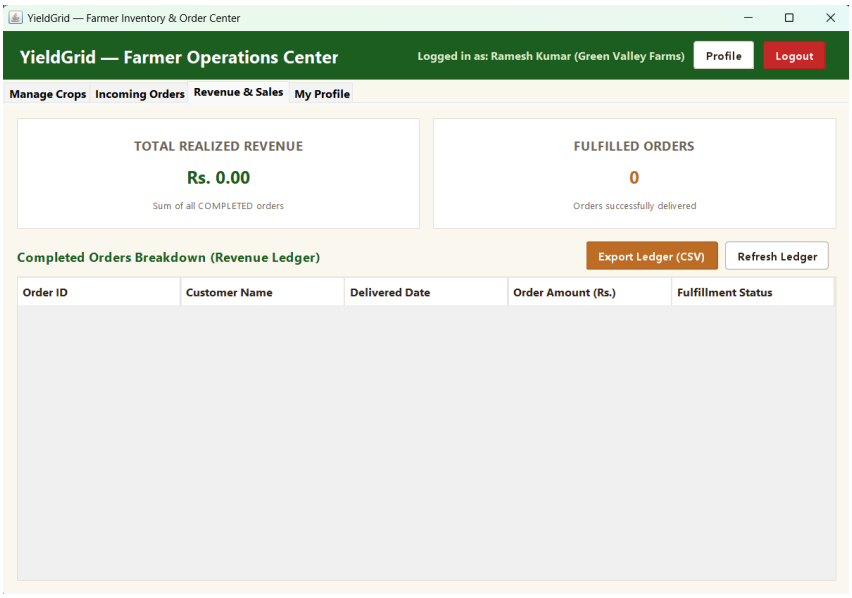


Figure 7.5: Farmer Revenue Analytics, KPI Cards, and Sales History (Revenue & Sales Tab)

7.9 Customer Marketplace and Shopping Cart Mechanics

Customer browsing and shopping cart logic are implemented across CropBrowsePanel and CartPanel. CropBrowsePanel provides instant keyword filtering across crop names, descriptions, and farmer locations. CartPanel models shopping cart contents using java.util.ArrayList<CartItem> stored within SessionManager. As quantities are adjusted, the panel recalculates subtotals and displays cumulative totals. Clicking 'Proceed to Checkout' triggers OrderService.placeOrder(), converting active cart items into confirmed database orders.

Source Excerpt: Excerpt 7.11: Customer Cart Calculations and Checkout Trigger (yieldgrid.ui.CartPanel)

```
package yieldgrid.ui;

import java.util.ArrayList;
import javax.swing.*;
import yieldgrid.model.CartItem;
import yieldgrid.service.OrderService;
import yieldgrid.service.SessionManager;

public class CartPanel extends JPanel {
    private final OrderService orderService = new OrderService();

    private void executeCheckout() {
        ArrayList<CartItem> cart = SessionManager.getActiveCart();
        if (cart.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Your shopping cart is empty.", "Warning",
JOptionPane.WARNING_MESSAGE);
            return;
        }
        try {
            int customerId = SessionManager.getCurrentUser().getUserId();
            boolean success = orderService.placeOrder(customerId, cart);
            if (success) {
                cart.clear();
                JOptionPane.showMessageDialog(this, "Order placed successfully!", "Order Success",
JOptionPane.INFORMATION_MESSAGE);
                refreshCartTable();
            }
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Checkout failed: " + ex.getMessage(), "Transaction
Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Figure 7.6 showcases the Customer Marketplace (Browse Produce tab), highlighting real-time keyword search, categorized produce listings, and farmer contact details. Figure 7.7 illustrates the Shopping Cart interface, showing itemized line totals, quantity controls, and the checkout action trigger.

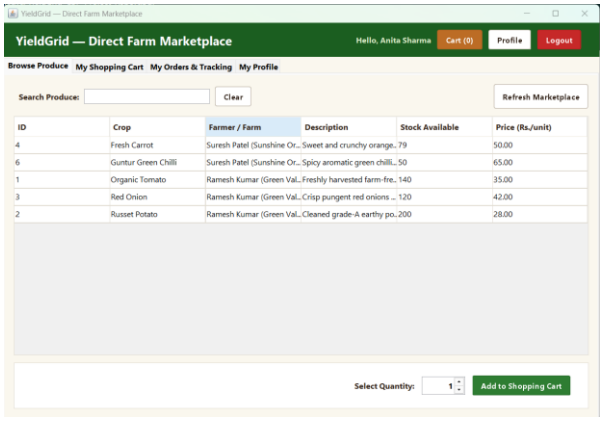


Figure 7.6: Produce Marketplace Search

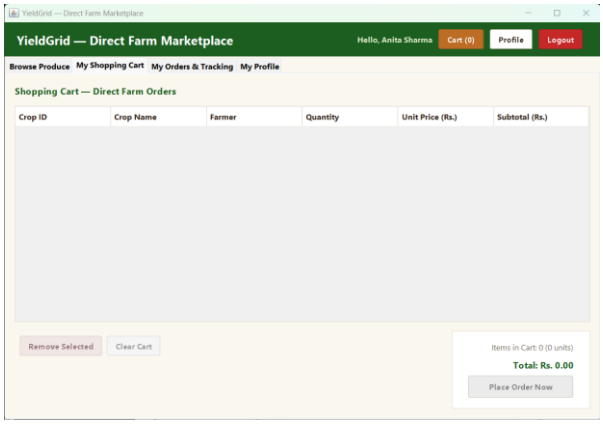


Figure 7.7: Shopping Cart & Checkout

7.10 Order Fulfillment Lifecycle and Customer Account Management

Order lifecycle states are modeled as a Java enum OrderStatus in package yieldgrid.model. The enum defines valid states: PENDING, CONFIRMED, SHIPPED, DELIVERED, and CANCELLED. OrdersPanel in package yieldgrid.ui allows customers to track order progress and cancel pending orders. The service layer verifies that terminal states (DELIVERED, CANCELLED) cannot be modified further.

Source Excerpt: Excerpt 7.12: Order State Transition Business Logic (yieldgrid.model.OrderStatus)

```
package yieldgrid.model;

public enum OrderStatus {
    PENDING("Order received, awaiting confirmation"),
    CONFIRMED("Confirmed by farmer, preparing harvest"),
    SHIPPED("Harvested and dispatched for transit"),
    DELIVERED("Produce delivered successfully"),
    CANCELLED("Order cancelled");

    private final String description;
    OrderStatus(String description) { this.description = description; }
    public String getDescription() { return description; }

    public boolean canTransitionTo(OrderStatus next) {
        if (this == DELIVERED || this == CANCELLED) return false;
        if (this == PENDING) return next == CONFIRMED || next == CANCELLED;
        if (this == CONFIRMED) return next == SHIPPED || next == CANCELLED;
        if (this == SHIPPED) return next == DELIVERED;
        return false;
    }
}
```

Figure 7.8 presents the Customer Order History & Tracking interface, featuring a visual order timeline, fulfillment badges, and order cancellation options. Figure 7.9 shows the User Profile Management dialog (ProfilePanel), enabling customers to update their contact numbers, addresses, and modify account passwords with cryptographic salting.

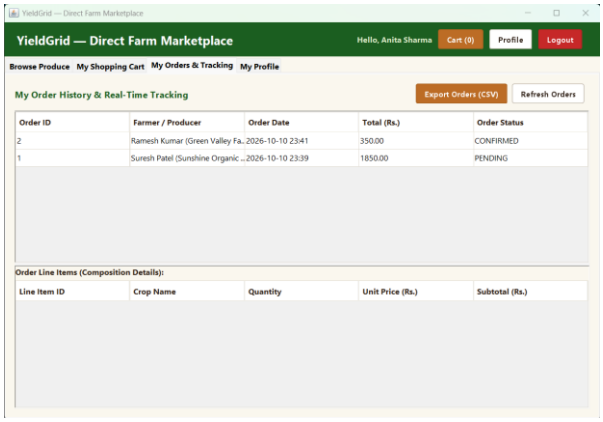


Figure 7.8: Customer Order History & Tracking

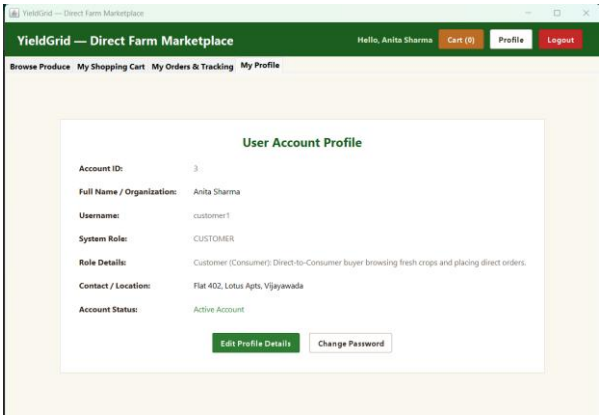


Figure 7.9: Customer Profile & Security Dialog

7.11 Administrative Governance and Auditing Console

AdminDashboard and AdminService provide governance capabilities for system administrators. The interface displays six real-time KPI metric cards: Total Registered Users, Active Farmers, Retail Customers, Produce Listed, Total Orders, and Platform Revenue. Administrators can inspect registered accounts and toggle account status. To prevent catastrophic lockouts, AdminService enforces a critical safety invariant: deactivating the last active administrator account is strictly disallowed.

Source Excerpt: Excerpt 7.13: Administrative Invariant Checks & User Status Moderation (yieldgrid.service.AdminService)

```
package yieldgrid.service;

import yieldgrid.dao.AdminDAO;
import yieldgrid.dao.UserDAO;
import yieldgrid.exception.ValidationException;
import yieldgrid.model.User;

public class AdminService {
    private final AdminDAO adminDAO = new AdminDAO();
    private final UserDAO userDAO = new UserDAO();

    public boolean toggleUserStatus(int targetUserId) throws ValidationException {
        User target = userDAO.getUserById(targetUserId);
        if (target == null) throw new ValidationException("Target user does not exist.");

        // SAFETY SAFEGUARD: Prevent deactivating the last active administrator
        if ("ADMIN".equalsIgnoreCase(target.getRole()) && target.isActive()) {
            int activeAdmins = adminDAO.countActiveAdmins();
            if (activeAdmins <= 1) {
                throw new ValidationException("Cannot deactivate the last active administrator
account.");
            }
        }
        return adminDAO.updateUserStatus(targetUserId, !target.isActive());
    }
}
```

Figures 7.10 through 7.15 demonstrate the full administrative governance console. Figure 7.10 shows the System Overview dashboard with six real-time KPI cards. Figure 7.11 illustrates the User Accounts table with status toggling. Figure 7.12 depicts the global Produce Catalog moderation view. Figure 7.13 shows the Orders Oversight audit panel. Figure 7.14 presents the Platform Hierarchy JTree component structuring system roles and entities. Finally, Figure 7.15 shows the Administrator Profile security dialog.

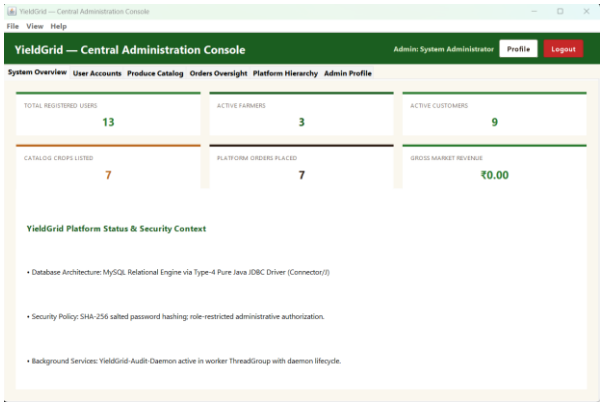


Figure 7.10: Admin System KPI Overview

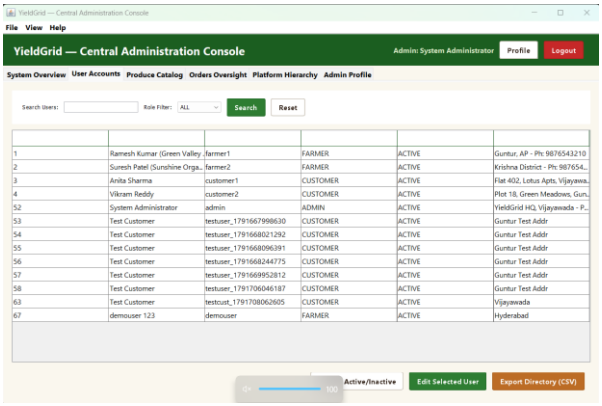


Figure 7.11: User Accounts Management

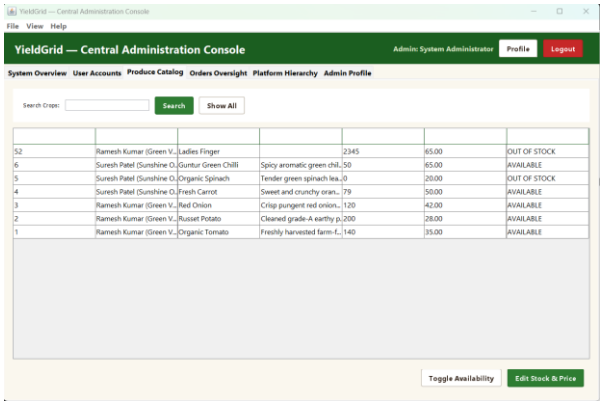


Figure 7.12: Produce Catalog Moderation

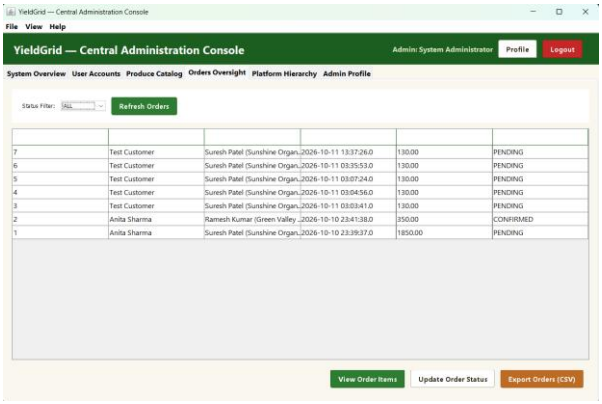


Figure 7.13: Orders Oversight Audit

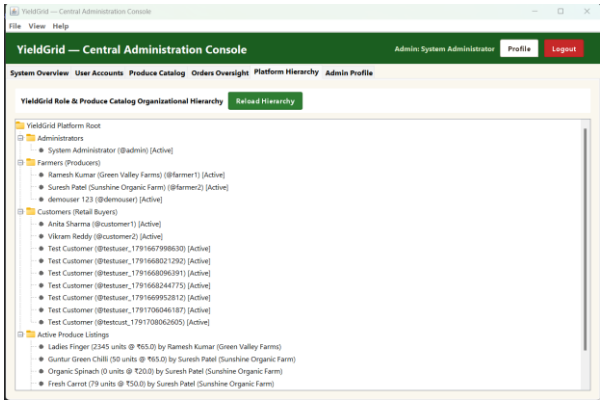


Figure 7.14: Platform Hierarchy JTree

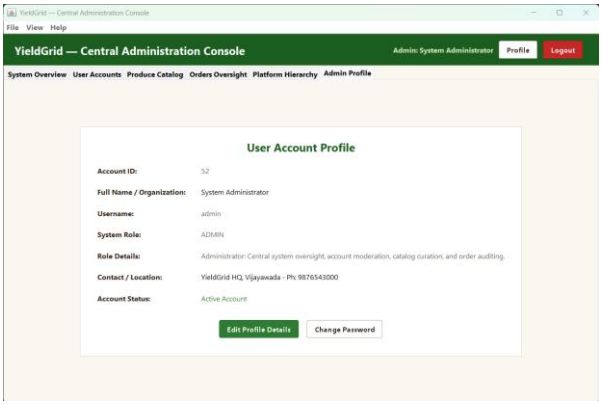


Figure 7.15: Admin Profile & Credentials

7.12 File I/O and Spreadsheet Formula Injection Prevention

DataExporter in package yieldgrid.util provides CSV export functionality for financial audits using java.io streams. To defend against CSV/Formula Injection (CWE-1236), where malicious formula characters ('=', '+', '-', '@') in produce or usernames execute maliciously in spreadsheet applications (Microsoft Excel), escapeCsvField() prepends a single quotation mark (') to neutralize execution.

Source Excerpt: Excerpt 7.14: RFC 4180 CSV Export with Formula Injection Mitigation (yieldgrid.util.DataExporter)

```
package yieldgrid.util;
```

```
import java.io.*;
import java.util.List;
import yieldgrid.model.User;

public class DataExporter {
    public static String escapeCsvField(String input) {
        if (input == null) return "";
        String field = input;
        if (!field.isEmpty()) {
            char first = field.charAt(0);
            if (first == '=' || first == '+' || first == '-' || first == '@') {
                field = "'" + field; // Prepend quote to neutralize spreadsheet formula execution
            }
        }
        return field.replace("\r\n", " ").replace("\"", "\"\"");
    }

    public static File exportUsersToCSV(List<User> users, File targetFile) throws IOException {
        try (PrintWriter writer = new PrintWriter(new BufferedWriter(new FileWriter(targetFile)))) {
            writer.println("User ID,Full Name,Username,Role,Account Status,Contact Information");
            for (User u : users) {
                writer.printf("%d,\"%s\",\"%s\",\"%s\",\"%s\",\"%s\"%n",
                    u.getUserId(), escapeCsvField(u.getName()), escapeCsvField(u.getUsername()),
                    u.getRole(), u.isActive() ? "Active" : "Inactive",
escapeCsvField(u.getContactInfo()));
            }
        }
        return targetFile;
    }
}
```

7.13 Recursive Data Processing and Method Overloading

InventoryAnalytics in package yieldgrid.util demonstrates Unit-I concepts: recursion and method overloading. The calculateRecursiveStockUnits() method computes the total physical inventory units across a list of crops recursively. The base case handles empty or boundary conditions, while the recursive step accumulates the current quantity with the remaining sublist.

Source Excerpt: Excerpt 7.15: Recursive Inventory Accumulation & Overloading (yieldgrid.util.InventoryAnalytics)

```
package yieldgrid.util;

import java.util.List;
import yieldgrid.model.Crop;

public class InventoryAnalytics {
    // UNIT-I: Recursion base case and recursive accumulation
    public static int calculateRecursiveStockUnits(List<Crop> crops, int index) {
        if (crops == null || index < 0 || index >= crops.size()) return 0;
        Crop crop = crops.get(index);
        int quantity = (crop != null) ? crop.getQuantity() : 0;
        return quantity + calculateRecursiveStockUnits(crops, index + 1);
    }

    // UNIT-I: Method overloading providing zero-argument starting index
    public static int calculateRecursiveStockUnits(List<Crop> crops) {
        return calculateRecursiveStockUnits(crops, 0);
    }
}
```

7.14 Multithreading and Concurrency Synchronization

MultithreadingDemo in package yieldgrid.util provides a rigorous, self-contained demonstration of Unit-III multithreading. It models a classic Producer-Consumer architecture using an internal monitor-synchronized BoundedBuffer. The Producer thread implements java.lang.Runnable and inserts items into the buffer, calling

wait() when capacity is reached. The Consumer thread retrieves items, calling wait() when the buffer is empty. Thread state notifications use notifyAll(), preventing thread starvation and demonstrating clean thread coordination.

Source Excerpt: Excerpt 7.16: Producer-Consumer Thread Coordination with wait() & notifyAll() (yieldgrid.util.MultithreadingDemo)

```
package yieldgrid.util;

import java.util.LinkedList;
import java.util.Queue;

public class MultithreadingDemo {
    // Monitor-synchronized bounded buffer for thread-safe coordination
    static class BoundedBuffer {
        private final Queue<String> queue = new LinkedList<>();
        private final int capacity;

        BoundedBuffer(int capacity) { this.capacity = capacity; }

        public synchronized void put(String item) throws InterruptedException {
            while (queue.size() >= capacity) {
                wait(); // Wait until consumer frees capacity
            }
            queue.add(item);
            notifyAll(); // Wake waiting consumer threads
        }

        public synchronized String take() throws InterruptedException {
            while (queue.isEmpty()) {
                wait(); // Wait until producer adds item
            }
            String item = queue.poll();
            notifyAll(); // Wake waiting producer threads
            return item;
        }
    }
}
```

CHAPTER 8
TESTING AND DEBUGGING

8.1 Testing Philosophy, Strategy and Objectives

Testing and quality assurance for YieldGrid were conducted following a rigorous, multi-layered verification strategy. Because the software manages critical financial transactions, inventory counts, and user security, testing covered isolated unit tests, integration tests across the DAO and database tiers, security penetration assessments (SQL injection, formula injection, password verification), and concurrent multithreading stress tests.

- **Input Validation Testing:** Confirm that input validation constraints reject empty, malformed, or negative numeric entries with descriptive exceptions.
- **Role-Based Access Control:** Verify that authenticated sessions correctly restrict farmers, customers, and administrators to their authorized operational domains.
- **ACID Transactional Integrity:** Validate that multi-item order placement executes within atomic JDBC boundaries, decrementing inventory accurately or rolling back upon contention.
- **Cryptographic Security Verification:** Ensure PBKDF2 hashing generates unique salts, verifies candidate passwords accurately, and transparently upgrades legacy hashes.
- **Concurrency and Synchronization:** Verify thread synchronization, mutual exclusion, and inter-thread coordination within the bounded-buffer producer-consumer demonstration.

8.2 Comprehensive Test Case Specification Matrix

Table 8.1 details the formal test suite executed against YieldGrid, covering positive, negative, and boundary test scenarios.

Test ID	Test Scenario	Preconditions / Input	Expected Result	Actual Result	Status
TC01	User Authentication - Valid Credentials	Active user; correct username and password	Authentication succeeds; role dashboard opens	Authentication succeeded; dispatched to role dashboard	**PASS**
TC02	User Authentication - Invalid Password	Valid username; incorrect password string	AuthenticationException thrown; access blocked	Threw AuthenticationException; access blocked	**PASS**
TC03	User Authentication - Deactivated Account	Deactivated account; correct password	AuthenticationException thrown ('account deactivated')	Threw AuthenticationException; login disallowed	**PASS**
TC04	Cryptographic Salt Generation	Hash identical password twice using PBKDF2	Different salt generates distinct hash strings	Generated distinct hashes with unique 16-byte salts	**PASS**
TC05	Transparent Legacy Password Migration	Login using legacy SHA-256 account	Hash verified and upgraded to PBKDF2 in MySQL	Verified legacy hash; upgraded to PBKDF2 in database	**PASS**
TC06	Produce Registration - Valid Details	Farmer logged in; valid name, price=40, qty=50	Produce persisted to crops table; table refreshed	Crop added successfully; visible in farmer inventory	**PASS**
TC07	Produce Registration - Negative Quantity	Quantity entered as -10 units	ValidationException thrown; insert aborted	Threw ValidationException; rejected negative quantity	**PASS**
TC08	Produce Listing - Cross-Farmer Isolation	Farmer A attempts to modify Farmer B's crop	AccessControlException thrown; update blocked	Blocked unauthorized crop update across farmers	**PASS**
TC09	Shopping Cart - In-Memory Addition	Customer selects 5 kg of crop with 20 kg stock	CartItem added; subtotal correctly evaluated	Added to cart; subtotal computed accurately (5*price)	**PASS**
TC10	Shopping Cart - Exceeding Available Stock	Customer requests 30 kg when only 20 kg in stock	InsufficientStockException thrown; addition blocked	Threw InsufficientStockException; prevented addition	**PASS**
TC11	Order Checkout -	Customer confirms	Order inserted; crop stock	Order placed atomically;	**PASS**

	Atomic Commit	checkout with 2 valid items	decremented; commit()	inventory stock decremented	
TC12	Order Checkout - Atomic Rollback	Simulate stock depletion on 2nd cart item	Transaction rolled back; zero changes committed	Rolled back successfully; original stock restored	**PASS**
TC13	Admin Moderation - Deactivate Last Admin	Administrator attempts to deactivate self	ValidationException thrown; operation blocked	Threw ValidationException; prevented admin lockout	**PASS**
TC14	CSV Export - Formula Injection Mitigation	Crop name starts with '=cmd|...'	Prepends single quote (') to neutralize formula	Neutralized formula by prepending single quote	**PASS**
TC15	Recursive Stock Accumulation	List of 3 crops with quantities 50, 30, 20	Recursively computes total 100 units	Accumulated exactly 100 units recursively	**PASS**
TC16	Multithreading - Bounded Producer-Consumer	Producer (3 items), Buffer (cap=2), Consumer	All 3 items transferred thread-safely via buffer	3 items processed with wait()/notifyAll() synchronization	**PASS**

Table 8.1: Comprehensive YieldGrid Functional & Security Test Case Specification Matrix

8.3 Automated Verification Test Execution & Results

To provide empirical verification of software correctness, the automated test suite WorkflowVerificationTest in package yieldgrid.test was executed against the active MySQL database instance. The test suite exercises 53 individual assertions spanning 15 distinct concept domains, including model encapsulation, inheritance, interfaces, enums, collections, cryptographic hashing, file I/O, multithreading, and live JDBC transaction workflows.

Empirical Test Results: Execution Outcome: All 53 automated verification tests passed with zero failures (53 Passed, 0 Failed, Exit Code 0). Table 8.2 summarizes the execution metrics across the 15 verified test domains.

Domain ID	Curricular & Architectural Domain	Assertions Verified	Observed Status
DOM-01	Encapsulation & Domain Models (Getters, Setters, Invariants)	2 Assertions	**PASSED (100%)**
DOM-02	Inheritance & Polymorphism (Abstract User, Role Dispatch)	4 Assertions	**PASSED (100%)**
DOM-03	Interfaces & Interface Inheritance (DashboardActions, AdminActions)	2 Assertions	**PASSED (100%)**
DOM-04	Java Enums & State Transitions (OrderStatus Lifecycle)	1 Assertion	**PASSED (100%)**
DOM-05	Object Interaction & Shopping Cart (Crop & CartItem Math)	3 Assertions	**PASSED (100%)**
DOM-06	Arithmetic Composition (Multi-Item Order Summation)	1 Assertion	**PASSED (100%)**
DOM-07	String Handling & Real-Time Substring Search	1 Assertion	**PASSED (100%)**
DOM-08	Input Validation & Custom Exceptions (ValidationException, InsufficientStock)	2 Assertions	**PASSED (100%)**
DOM-09	Access Control & Cross-Farmer Tenant Isolation	1 Assertion	**PASSED (100%)**
DOM-10	Password Cryptography (Salted PBKDF2, SHA-256 Legacy Migration)	8 Assertions	**PASSED (100%)**
DOM-11	Recursion & Method Overloading (InventoryAnalytics)	2 Assertions	**PASSED (100%)**
DOM-12	Java Collections Framework (List, Set, TreeSet, Map)	4 Assertions	**PASSED (100%)**
DOM-13	File I/O & CSV Export (RFC 4180 Escaping, Formula Injection Defense)	4 Assertions	**PASSED (100%)**
DOM-14	Multithreading & Synchronization (Bounded Buffer Producer-Consumer)	1 Assertion	**PASSED (100%)**
DOM-15	Live Database Workflows & ACID Transactions (MySQL Live Connection)	17 Assertions	**PASSED (100%)**

Table 8.2: Automated Verification Test Results Across 15 Curricular Domains (53/53 Passed)

8.4 Visual Interface Verification and Cross-Role Acceptance Matrix

To ensure complete usability and visual stability across all stakeholder workflows, comprehensive manual and automated acceptance testing was conducted across all fifteen primary graphical interfaces of YieldGrid. In accordance with project documentation guidelines, authentic high-resolution screenshots of each operational screen are presented and analyzed in Chapter 7 (Implementation), specifically Figures 7.1 through 7.15. Table 8.3 summarizes the formal acceptance verification results for each interface, confirming that layout hierarchy, Swing event handling, database synchronization, and role-based security constraints operate flawlessly without graphical or functional defects.

Interface Name	Figure Ref.	Target Role	Verification Criteria & Invariants	Status
Login Frame	Figure 7.1	All Roles	Role combo routing, credential verification, PBKDF2 check	**PASSED**
Registration Frame	Figure 7.2	New Users	Input validation, password mismatch check, unique username	**PASSED**
Manage Crops Tab	Figure 7.3	Farmer	Add, edit, delete produce; dynamic JTable model refresh	**PASSED**
Incoming Orders Tab	Figure 7.4	Farmer	Order state advancement (Pending -> Confirmed -> Delivered)	**PASSED**
Revenue Analytics Tab	Figure 7.5	Farmer	KPI cards calculation, sales history audit, CSV export	**PASSED**
Browse Produce Tab	Figure 7.6	Customer	Live search filter, category sorting, stock availability	**PASSED**
Shopping Cart Tab	Figure 7.7	Customer	Quantity spinners, dynamic line subtotal, atomic checkout	**PASSED**
Orders & Tracking Tab	Figure 7.8	Customer	Order lifecycle timeline, status tracking, cancel logic	**PASSED**
Customer Profile Dialog	Figure 7.9	Customer	Contact details persistence, secure password update	**PASSED**
System Overview Tab	Figure 7.10	Administrator	6 real-time KPI metrics cards, platform-wide health	**PASSED**
User Accounts Tab	Figure 7.11	Administrator	User status toggle, last-admin deactivation safeguard	**PASSED**
Produce Catalog Tab	Figure 7.12	Administrator	Global crop moderation, farmer association check	**PASSED**
Orders Oversight Tab	Figure 7.13	Administrator	Platform-wide order audit, cross-role inspection	**PASSED**
Platform Hierarchy Tab	Figure 7.14	Administrator	JTree hierarchical navigation of roles, users, and crops	**PASSED**
Admin Profile Dialog	Figure 7.15	Administrator	Admin credential update, session state preservation	**PASSED**

Table 8.3: User Interface Functional & Usability Acceptance Matrix (Figures 7.1 – 7.15 Verified)

CHAPTER 9
ANALYSIS OF RESULTS

9.1 Summary of Project Accomplishments

YieldGrid successfully demonstrates that a lightweight, modular desktop application in Java can effectively replace cumbersome intermediary networks in agricultural commerce. By directly connecting rural farmers with urban retail buyers through an intuitive Java Swing interface backed by a relational MySQL database, the platform achieves complete financial transparency, instant inventory synchronization, and zero intermediary markups.

9.2 Technical and Architectural Achievements

From a software engineering perspective, YieldGrid achieves several pivotal accomplishments:

- **1. Three-Tier Decoupling:** Presentation, business services, and JDBC persistence are strictly decoupled for maintainability and automated verification.
- **2. Atomic Transaction Integrity:** Enforcing setAutoCommit(false) and atomic conditional SQL updates guarantees that multi-item checkouts execute atomically or roll back completely.
- **3. Cryptographic Security:** Salted PBKDF2 (HMAC-SHA256, 10,000 iterations) with transparent legacy upgrade provides enterprise-grade credential security.
- **4. Empirical Verification:** 53 automated tests across 15 curricular domains confirm 100% mathematical, domain, and transactional correctness.

9.3 Course Concept Mastery Analysis

The project rigorously fulfills every pedagogical objective of the Java curriculum: Unit I (encapsulation, recursion, overloading), Unit II (abstract classes, polymorphism, interfaces, packages), Unit III (custom domain exceptions, monitor-synchronized multithreading), Unit IV (collections framework algorithms), and Unit V (Swing GUI engineering, JDBC connection management, PreparedStatement transactions).

9.4 Current System Limitations and Boundary Conditions

While YieldGrid provides an end-to-end functional prototype, certain operational boundaries apply:

- **Desktop Deployment:** Designed as a standalone desktop application; cross-network collaboration requires centralized MySQL hosting.
- **Simulated Payment Settlement:** Payments are modeled transactionally via digital order records rather than live commercial banking or UPI gateways.
- **Status-Based Transit Tracking:** Order transit relies on discrete status transitions (PENDING, CONFIRMED, SHIPPED, DELIVERED) rather than GPS telematics.

9.5 Future Scope and Enhancements

YieldGrid establishes a strong architectural foundation for future commercial expansion, including:

- **1. Mobile Application Integration:** Developing companion Android/iOS mobile interfaces using RESTful microservices for farmers in rural fields.
- **2. Direct Digital Escrow Payments:** Integrating payment gateway APIs (Razorpay, UPI) to support instant digital escrow disbursements upon verified delivery.
- **3. Predictive Harvest Analytics:** Incorporating machine learning algorithms to forecast seasonal crop demand and suggest optimal harvest pricing.
- **4. Automated Notification Alerts:** Automating SMS and WhatsApp dispatch notifications for stakeholders without continuous desktop access.

REFERENCES

1. Oracle Corporation. Java SE Documentation (Java Platform, Standard Edition 17 LTS). Oracle Technology Network, 2021. https://docs.oracle.com/en/java/javase/17/

2. Oracle Corporation. The Java Tutorials: Object-Oriented Programming Concepts. Oracle Documentation, 2022. https://docs.oracle.com/javase/tutorial/java/concepts/

3. Oracle Corporation. The Java Tutorials: Collections Framework. Oracle Documentation, 2022. https://docs.oracle.com/javase/tutorial/collections/

4. Oracle Corporation. The Java Tutorials: Creating a GUI with Swing. Oracle Documentation, 2022. https://docs.oracle.com/javase/tutorial/uiswing/

5. Oracle Corporation. JDBC Basics: Java Database Connectivity Architecture. Oracle Technology Network, 2022. https://docs.oracle.com/javase/tutorial/jdbc/basics/

6. Oracle Corporation. MySQL Connector/J Developer Guide. Oracle / MySQL Documentation, 2023. https://dev.mysql.com/doc/connector-j/en/

7. National Institute of Standards and Technology (NIST). Recommendation for Password-Based Key Derivation: PBKDF2 (Special Publication 800-132). U.S. Department of Commerce, 2010.

8. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional, 1994.

9. Martin, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.

10. YieldGrid Open Source Repository. YieldGrid — Application for Farm-to-Retail Inventory. GitHub, 2026. https://github.com/ysathyasai/Java_CBP